Unidade 1

Apresentação

O crescimento da Internet, dos dispositivos móveis e das aplicações digitais provocou um aumento expressivo na quantidade de dados gerados e armazenados em todo o mundo. Além do grande volume, esses dados apresentam formatos cada vez mais variados, menos estruturados e distribuídos, criando novos desafios para seu armazenamento, processamento e análise.

Nesse cenário, as estruturas tradicionais dos bancos de dados relacionais passaram a apresentar limitações para determinadas aplicações, especialmente aquelas que precisam lidar com grandes volumes de dados, alta escalabilidade e diferentes formatos de informação. Como alternativa, surgiram os bancos de dados NoSQL, desenvolvidos para atender necessidades que nem sempre são solucionadas de forma adequada pelos modelos relacionais.

Nesta Unidade de Aprendizagem, você conhecerá a origem e a evolução dos bancos de dados NoSQL, compreenderá suas principais características e diferenças em relação aos bancos relacionais e identificará os diferentes tipos de bancos não relacionais e suas aplicações. Também serão abordados os princípios do modelo de consistência BASE, importante para a compreensão do funcionamento dessas tecnologias.

Bons estudos.

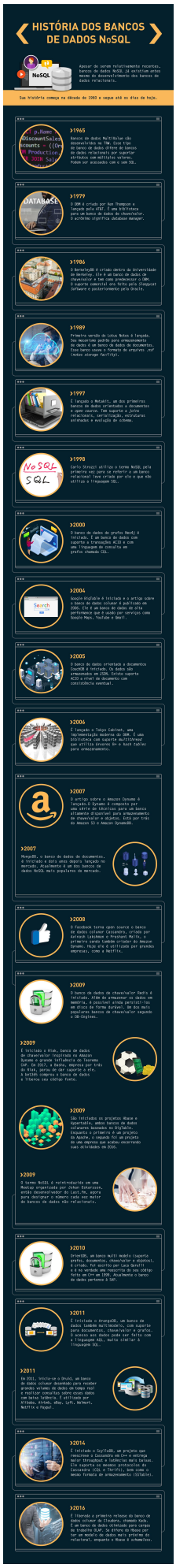
Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Descrever a origem, a evolução e os principais conceitos dos bancos de dados NoSQL.
- Identificar os principais tipos e características dos bancos de dados não relacionais.
- Explicar os princípios do modelo de consistência BASE.
- Diferenciar os modelos de bancos de dados relacionais (SQL) e não relacionais (NoSQL).
- Demonstrar a aplicabilidade dos diferentes tipos de bancos de dados NoSQL.

Infográfico: História dos bancos de dados não relacionais

A história dos bancos de dados não relacionais, por incrível que pareça, começa antes mesmo da criação do modelo relacional. Ao longo dos anos, diversas tecnologias foram criadas e deram início ao que hoje se conhece como bancos de dados NoSQL.

Neste Infográfico, você verá a evolução dessas tecnologias e quais foram os precursores do movimento NoSQL.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Conteúdo do Livro: História dos bancos de dados não relacionais

Com a globalização, a evolução tecnológica e, principalmente, a Internet, o volume de dados produzidos pela sociedade aumenta constantemente. Com isso, existe a necessidade das empresas terem capacidade de processar e armazenar grandes volumes de dados. Bancos de dados NoSQL surgiram para lidar com esse problema, quebrando alguns paradigmas definidos pelos bancos de dados relacionais.

No capítulo História dos bancos de dados não relacionais, da obra *Banco de Dados Não Relacional*, base teórica desta Unidade de Aprendizagem, você conhecerá a história dos bancos de dados NoSQL, as razões do seu surgimento e a sua evolução ao longo dos anos, bem como uma descrição detalhada de alguns dos principais bancos de dados NoSQL.

Boa leitura.

BANCO DE DADOS NÃO RELACIONAL

Maycon Bordin



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

História dos bancos de dados não relacionais

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever a história dos bancos de dados não relacionais.
- Explicar a evolução das tecnologias relacionadas a bancos de dados NoSQL.
- Identificar os principais bancos de dados NoSQL existentes desde o seu surgimento até a atualidade.

Introdução

Os bancos de dados são o alicerce da maioria dos sistemas que utilizamos no nosso dia a dia. Com o avanço das tecnologias, a importância dos dados está cada vez maior, bem como a procura por formas mais eficientes de se armazenar e processar grandes volumes de dados. Assim, conhecer a história dos bancos de dados não relacionais (NoSQL) é fundamental para quem deseja trabalhar com *big data*.

Neste capítulo, você conhecerá a história dos bancos de dados não relacionais, desde o seu surgimento até os momentos atuais, com a criação e a popularização do termo NoSQL, bem como a ascensão do *big data*. Além disso, conhecerá as principais tecnologias por trás dos principais bancos de dados não relacionais. Por fim, verá quais são os bancos de dados mais relevantes. Os conceitos que serão vistos neste capítulo servirão de base para a compreensão não só dá história dos bancos de dados NoSQL, mas também de como definir qual o melhor banco de dados para determinadas situações práticas, os tipos de *trade-offs* que deverão ser tomados e as consequências dessas decisões com relação a vantagens, desvantagens e limitações de cada banco de dados NoSQL.

1 Histórico

É impossível falar de bancos de dados não relacionais (NoSQL) sem antes mencionar os seus principais concorrentes, os bancos de dados relacionais. Os **bancos de dados relacionais** surgiram a partir do modelo relacional, desenvolvido por Codd em 1970 (CODD, 1970), e, até os tempos atuais, são os bancos de dados mais utilizados no mercado. Eles são conhecidos por serem sistemas de propósito geral, utilizarem a linguagem SQL (Structured Query Language; ou Linguagem de Consulta Estruturada, em português) para consulta e manipulação de dados e estruturas de dados, além das propriedades ACID para transações.

De acordo com Pritchett (2008), o acrônimo ACID reflete as características listadas a seguir.

- **Atomicidade:** as transações são atômicas, executam completamente ou não executam.
- **Consistência:** as transações criam novos estados válidos, ou, em caso de falha, o banco de dados volta para o último estado válido.
- **Isolamento:** garante que uma transação em andamento não será interferida por outras transações.
- **Durabilidade:** os dados estarão disponíveis na sua forma correta mesmo se o sistema falhar ou reiniciar.

Outra característica marcante dos bancos de dados relacionais é a integridade referencial, que garante a acurácia e a consistência dos dados dentro de um relacionamento entre tabelas. Isso é feito por meio de uma chave estrangeira, que faz referência a um valor de uma chave primária em outra tabela, e a integridade referencial garantirá que esse relacionamento é íntegro (o registro referenciado existe) (CHAPPLE, 2020).

Além disso, os bancos de dados relacionais são caracterizados pela normalização dos seus dados, um processo que busca otimizar os bancos de dados, além de garantir a maior consistência destes, evitando a duplicação de informações e as dividindo em tabelas de acordo com o tipo de elemento que está sendo armazenado (KOKAY, 2012). Desse modo, as tabelas se relacionam por meio do uso de chaves estrangeiras, que garantem a consistência das informações.

Essas características regem os bancos de dados relacionais até os dias de hoje, porém, ao longo dos anos, limitações foram sendo encontradas. Segundo Leavitt (2010), dentre as limitações de bancos de dados relacionais, as principais são: o fato de que esses sistemas não foram construídos para aplicações distribuídas; o cruzamento de dados através de *joins* tende a custar caro em termos de requisitos de *hardware* necessários; e o modelo de dados disponível é limitado, não havendo muita flexibilidade em casos em que a estrutura de dados não é completamente conhecida. Além disso, é difícil escalar esses sistemas de forma horizontal, de modo que a escalabilidade vertical (aumento de recursos de *hardware*) é a opção mais simples tecnicamente, porém é a mais custosa e, em muitos casos, pode envolver a aquisição de *hardware* proprietário.

Na metade da década de 90, com o ganho de popularidade da internet, os bancos de dados relacionais não foram capazes de acompanhar a escala das aplicações. Isso levou ao surgimento dos bancos de dados não relacionais, capazes de acompanhar a escala da internet, além de serem capazes de armazenar dados não estruturados e os processar de forma mais rápida. Os bancos de dados não relacionais também são chamados de NoSQL, termo criado em 1998 por Carlo Strozzi para se referir a um banco de dados relacional leve e *open source* que não utilizava a linguagem SQL (KUZNETSOV; POSKONIN, 2014). Contudo, o termo só ganhou a conotação que tem hoje em 2009, quando Eric Evans e Johan Oskarsson o utilizaram para se referir a bancos de dados não relacionais (STRAUCH, 2011). Ele pode ser interpretado de forma literal como "não SQL" (ausência de linguagem SQL) ou como "Not Only SQL" (Não Apenas SQL), apesar de Sadalage e Fowler (2019) recomendarem apenas o uso de NoSQL, pois até mesmo bancos de dados tradicionais, como Oracle e Postgres, poderiam se enquadrar nessa classificação.

Os bancos de dados NoSQL são caracterizados pela ausência da linguagem SQL, apesar de alguns deles apresentarem linguagens similares. Esses bancos de dados geralmente são *open source*, pois foram desenvolvidos para funcionar em *clusters* de computadores, de modo que oferecem diversas variações de consistência de dados. Além disso, eles têm como característica a ausência de esquema, permitindo a adição de novas colunas de forma livre (SADALAGE; FOWLER, 2019).

Apesar de o termo ser relativamente novo e englobar em sua classificação bancos de dados que são, em sua maioria, de código aberto, desenvolvidos no século XXI e que não utilizam a linguagem SQL, os bancos de dados não relacionais surgiram muito antes (SADALAGE; FOWLER, 2019). Os seus precursores datam da década de 60, visto que um dos primeiros bancos de dados não relacionais surgiu em 1965 na TRW, chamado de MultiValue, um banco de dados multidimensional com suporte a atributos que suporta uma lista de valores, em vez de apenas um valor (HAUGEN, 2010). Em 1979, o DBM foi lançado pela AT&T, um dos primeiros bancos de dados de chave/valor. Em 1989, surgiu o Lotus Notes, um *software* que utiliza como armazenamento interno um banco de dados orientado a documentos.

O banco de dados em grafo Neo4j surgiu no ano 2000. Em 2004, o Google BigTable foi iniciado, um banco de dados colunar distribuído, construído em cima do sistema de arquivos do Google (GFS ou Google File System). Em 2005, foi iniciado o CouchDB, outro banco de dados orientado a documentos. Em 2007, a Amazon apresentou o Dynamo, um sistema distribuído para armazenamento de dados altamente disponível que, depois, serviria de base para o Amazon S3. É importante ressaltar a relevância que a criação do BigTable (Google) e do Dynamo (Amazon) teve no mundo dos bancos de dados. De acordo com Sadalage e Fowler (2019), esses artigos levaram à criação de diversos bancos de dados NoSQL, que experimentavam formas diferentes de armazenamento de dados e passaram a ser um assunto importante nas principais conferências da área.

Em 2007, também surgiu o MongoDB, um banco de dados orientado a documentos. Em 2008, o Facebook abriu o código do banco de dados colunar Cassandra. Em 2009, surgiram dois clones *open source* do banco de dados colunar BigTable, o HBase e o Hypertable. O primeiro é *open source* e desenvolvido pela Apache, ao passo que o segundo foi criado por uma empresa que não existe mais. Dentre os bancos de dados NoSQL mais recentes, estão o banco de dados colunar Apache Druid, mais voltado para o armazenamento de dados em tempo real, e o banco de dados de grafos Apache Giraph, ambos lançados em 2011. Em 2016, foi lançada uma variante do BigTable, chamada de Apache Kudu.



Fique atento

A Google foi um dos precursores das principais tecnologias que hoje servem de base para o que se conhece como *big data*. É o caso do Hadoop e do HDFS, que foram inspirados nos artigos da Google descrevendo o MapReduce e o Google File System.

2 Evolução das tecnologias de bancos de dados NoSQL

O desafio de se armazenar e processar grandes volumes de dados com formatos diversos levou ao surgimento de bancos de dados NoSQL. Para conseguir atender a essas demandas, foi necessário abrir mão de características existentes em bancos de dados relacionais, a fim de dar lugar a outras características mais flexíveis. Isso porque lidar com grandes quantidades de dados de forma estável e permitindo que aplicações escalem a um custo baixo significa que características como a integridade garantida através de transações e a flexibilidade na criação de índices e consultas nem sempre serão possíveis (TIWARI, 2011).

Apesar de, em alguns casos, a ausência de algumas das propriedades ACID ser inaceitável, Strauch (2011) afirma que existem casos em que a redução da complexidade do banco de dados pode ser favorável, dispensando mecanismos complexos que garantiriam propriedades que nem sempre são necessárias. A perda dessas características dá lugar à capacidade de processar grandes volumes de dados em curtos espaços de tempo. Esse é o caso do Google, que é capaz de processar 20 petabytes de dados armazenados no Bigtable através do MapReduce.

Outra vantagem dessa classe de bancos de dados é que eles foram criados para escalar, ou seja, eles são capazes de responder a mais requisições e comportar volumes maiores de dados por meio da adição de um *hardware* ou da adição de novas máquinas em um *cluster*. Dessa forma, essas soluções geralmente escalam horizontalmente com maior facilidade e com um *hardware* padrão (máquinas normais com custo reduzido em relação a servidores de aplicação específicos), permitindo o *sharding* dos dados com um esforço muito menor do que o necessário para bancos de dados relacionais e sem o alto custo envolvido em bancos de dados proprietários (STRAUCH, 2011).

Além disso, os bancos de dados NoSQL, em muitos casos, tornam desnecessário o uso de mecanismos para mapeamento dos dados armazenados no banco de dados, para que eles possam se adequar a linguagens orientadas a objetos, como ORMs (Object-Relational Mapping; ou Mapeamento Objeto-Relacional, em português) para bancos de dados relacionais. Esse problema é descrito por Sadalage e Fowler (2019) como **incompatibilidade de impendância**, pois ou os dados possuem estruturas muito simples e dificilmente se beneficiariam das funcionalidades fornecidas por bancos relacionais e pelo SQL, ou a estrutura dos dados é muito semelhante à orientação a objetos.

Grandes empresas da internet são proeminentes no uso das tecnologias NoSQL, e a Google e a Amazon, por meio de artigos publicados descrevendo as suas alternativas aos bancos de dados relacionais, foram as propulsoras desse movimento.



Saiba mais

Sharding é uma técnica para dividir um conjunto de dados entre vários computadores de um *cluster* de banco de dados. Essa técnica permite que cada computador seja responsável por uma parcela de todos os dados, aumentando, assim, a capacidade do banco de dados de responder a requisições e de armazenar dados.

Hoje, o uso de bancos de dados NoSQL é largamente difundido, principalmente em empresas de tecnologia, devido, em parte, ao baixo custo de implementação e à alta escalabilidade. Em contrapartida, tecnologias mais tradicionais de bancos de dados de alto desempenho (p. ex., Oracle RAC [Real Application Cluster]) são utilizadas apenas por empresas com grande poder aquisitivo e que geralmente estão presas a tecnologias proprietárias ou não podem se arriscar a utilizar bancos de dados NoSQL.

A seguir, serão descritas algumas das técnicas e dos conceitos que permitiram a criação e a evolução dos bancos de dados NoSQL.

Teorema CAP

Em sistemas distribuídos, observaram-se algumas características desejáveis, bem como algumas restrições com relação a essas características e a necessidade de se fazer um *trade-off* entre elas. Introduzido por Eric Brewer, esse *trade-off* é chamado de **teorema CAP** (GILBERT; LYNCH, 2012) A seguir, são listados os significados do acrônimo CAP (SADALAGE; FOWLER, 2019).

- ***Consistency* (consistência):** indica se e quando um sistema está em um estado consistente após a execução de uma operação. Por exemplo, se uma atualização é feita em um escritor do sistema e todos os leitores do sistema conseguem ver essa atualização imediatamente, o sistema é consistente.
- ***Availability* (disponibilidade):** o sistema deve permanecer disponível mesmo após a falha de nós ou a indisponibilidade de algum *hardware* ou *software*.
- ***Partition tolerance* (tolerância a falhas de partição):** habilidade de um sistema de se manter operante mesmo com partições na rede, isolando alguns nós em ilhas. Pode significar, ainda, a capacidade de um sistema de lidar com a adição ou a remoção de nós.

Segundo o teorema CAP, um sistema distribuído só pode ter duas das três propriedades descritas, gerando, assim, três combinações possíveis de sistemas (ZVAREVASHE; GOTORA, 2014), descritas a seguir.

1. **CA:** são sistemas que garantem a consistência dos dados e possuem alta disponibilidade, mas não suportam falhas de partição. Por exemplo: Vertica, Greenplum.
2. **CP:** são sistemas que garantem a consistência dos dados e são tolerantes a falhas de partição, porém são bancos de dados que não têm boa disponibilidade. Esses sistemas possuem as propriedades ACID. Por exemplo: Google BigTable, Hypertable, HBase, MongoDB.
3. **AP:** são sistemas que estão sempre disponíveis, mas podem apresentar inconsistências nos dados. Esses sistemas possuem as propriedades BASE. Por exemplo: Voldemort, Tokyo Cabinet, CouchDB, Riak.

Consistência eventual

Com o advento de sistemas largamente distribuídos, garantias fortes de consistência passaram a ser deixadas de lado, dando espaço para modelos mais fracos, sendo a consistência eventual o modelo mais notável (BAILIS; GHODSI, 2013). A consistência eventual fornece algumas garantias: caso não haja mais nenhuma atualização em determinado dado, todas as leituras para esse dado, eventualmente, retornarão o mesmo valor. Esse modelo não descarta a possibilidade de o cliente ler um dado inconsistente, mas garante que, em algum momento do futuro, eventualmente o dado estará consistente. Uma das vantagens da consistência eventual é a possibilidade de criar sistemas de larga escala capazes de continuar operando mesmo na presença de falhas. Ou seja, eles estão sempre disponíveis, mesmo que isso signifique que uma parcela dos usuários verá dados desatualizados por um determinado período de tempo.

Segundo Bailis e Ghodsi (2013), os sistemas eventualmente consistentes geralmente propagam atualizações de forma assíncrona, garantindo que, em algum momento, todo o sistema convergirá para um estado consistente, ou seja, todos os nós terão a mesma cópia do dado atualizada. A propagação de atualizações, que podem ocorrer de forma concorrente para um mesmo dado, implica o uso de mecanismos para garantir que os nós de um sistema consigam saber qual é a atualização mais nova que deve ser aplicada. Um desses mecanismos é o uso de um valor de relógio atrelado a cada escrita, permitindo, assim, o uso de uma regra simples, como a *last write wins* (a última escrita ganha). Entretanto, esse mecanismo requer que os relógios de todos os nós estejam sincronizados para que as escritas sejam aplicadas corretamente; isso pode ser alcançado (CORBETT *et al*., 2013) ou contornado por outras técnicas, como é o caso dos Vector Clocks.

Outro problema que pode surgir com a consistência eventual é que, se um cliente efetua uma atualização, dependendo do sistema, existe uma chance de que ele não veja as suas atualizações instantaneamente. Isso pode gerar alguns problemas, pois o cliente pode pensar que a sua atualização não ocorreu e a enviar novamente. Contudo, esse problema pode ser resolvido através do princípio RYOW (Read Your Own Writes; ou Leia as suas Próprias Escritas, em português), que afirma que os clientes devem ver as suas atualizações imediatamente, mesmo que a atualização ocorra em um servidor, e a leitura, em outro (FAOUR, 2018). Outra forma de se conseguir isso é através da consistência de sessão, ou seja, um cliente é associado a um único servidor, garantindo, assim, que suas atualizações estejam visíveis a ele.

Vector Clocks

Segundo Prakash e Singhal (1997), o Vector Clocks é um mecanismo para controlar a dependência entre eventos causais de processos diferentes, os quais não dependem de relógios sincronizados entre os nós, não necessitam de uma ordem total de revisões para causar *reasoning* e não precisam armazenar e manter várias revisões de um dado em vários nós. Os Vector Clocks podem ser utilizados para a propagação de estado entre os nós através de protocolos Gossip (fofoca) ou epidêmicos, este podendo operar tanto pelo modelo de transferência de estado (State Transfer Model) como pelo de operações (Operation Transfer Model).

Os protocolos Gossip conseguem entregar apenas consistência eventual, visto que um *cluster* com *n* nós irá demorar O(log n) rodadas para propagar as suas atualizações. Em contrapartida, esse método fornece escalabilidade ao sistema e evita pontos únicos de falha (SPoF, Single Point of Failure). De acordo com Shapiro *et al.* (2011), além dos protocolos Gossip, existem outros métodos para propagação de atualizações para réplicas, como o método de antientropia.

A consistência eventual do protocolo Gossip levanta ainda outras questões, já que um sistema pode ter nós em estados diferentes ao mesmo tempo, exigindo algum tipo de versionamento dos dados e alguma forma de comunicar as atualizações para outros nós.

Sharding

A escalabilidade de bancos de dados NoSQL segue a estratégia de escalabilidade horizontal para passar pela clusterização de servidores de banco de dados, o uso de réplicas e o *sharding* dos dados. A utilização de *cache* em memória também pode ser adotada como estratégia para o armazenamento de dados que podem estar distribuídos, removendo parte da carga do banco de dados principal. Contudo, essa solução também apresenta limites de escalabilidade.

As técnicas listadas anteriormente também são utilizadas em bancos de dados relacionas, e, nos bancos de dados NoSQL, existem desafios com relação ao *sharding* de dados, principalmente no que diz respeito ao mapeamento dos dados em servidores (STRAUCH, 2011). Essa função é feita normalmente através do *hashing*, fazendo uso, por exemplo, da chave primária dos objetos para decidir em qual servidor o dado será armazenado:

partition = hash(o) mod n with o = object to hash, n = number of nodes

A fórmula apresentada é um exemplo de como a escolha da partição onde o objeto será armazenado pode ser realizada. Nesse caso, é gerada uma *hash* a partir do objeto a ser armazenado e efetuada a operação de módulo utilizando o número de nós disponíveis no *cluster*. Entretanto, essa solução apresenta um problema, pois ela não leva em consideração que os nós do *cluster* podem aparecer e desaparecer a qualquer momento (falha no servidor, na rede, no banco de dados), tornando alguns dados indisponíveis.

Para resolver esse problema, pode-se utilizar a técnica do *hash* consistente, em que cada nó do sistema recebe da função de *hashing* um intervalo determinado, e todos os objetos que se enquadrarem nesse intervalo serão ali armazenados. Esses intervalos são muito flexíveis, de modo que, se um nó sai do sistema, outros nós tomarão conta do seu intervalo, da mesma forma que, se um novo nó se junta ao sistema, ele receberá parte do intervalo de outro nó. Isso permite que os clientes calculem em que nó determinado dado está sem a necessidade de um servidor de metadados, como ocorre no Google File System (GFS).

A Figura 1, a seguir, apresenta um exemplo de *hashing* consistente, em que o anel à esquerda representa o estado inicial de um sistema com três nós (A, B e C) e quatro objetos (1, 2, 3, 4). O anel funciona em sentido horário, ou seja, cada objeto no anel está armazenado no nó à sua esquerda. No anel à direita, o nó C saiu por algum motivo, ao passo que o anel D entrou no sistema. Este último tomou metade dos objetos do nó A (um) e passou a cuidar dos objetos do nó C.

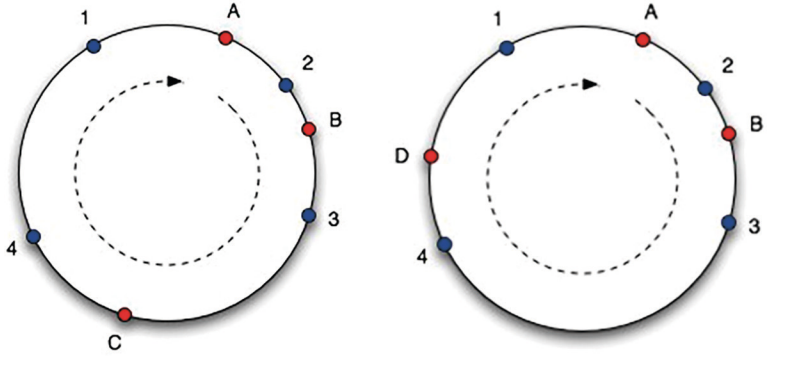


Figura 1. Exemplo de *hashing* consistente, antes e depois da entrada e saída de um nó.
Fonte: Strauch (2011, p. 40).

Sem muito esforço, é possível observar a falta de balanceamento na distribuição dos objetos entre os nós. Para superar isso, foi introduzido o conceito de **nós virtuais**, em que cada nó real pode ter um ou mais nós virtuais, de acordo com a sua capacidade de *hardware*, e são os nós virtuais que receberão os intervalos de *hashing*. Ainda assim, o problema de indisponibilidade de dados no caso da saída de um nó persiste. Para resolver esse problema, foi introduzido o fator de replicação (r), que determina que não apenas um nó físico fique responsável por um objeto, mas *r* nós físicos.

A natureza dinâmica dos nós em um sistema distribuído só pode ser alcançada por meio da comunicação entre eles. Cada vez que um novo nó se junta ao sistema, ele precisa anunciar a sua chegada aos demais nós (ou adjacentes), para que estes ajustem as suas propriedades de dados, cedendo parte para o nó recém-chegado, que pode os copiar de só uma vez ou de forma assíncrona. Já quando um nó sai do sistema, geralmente ele não dá um aviso prévio (dependendo da situação que levou à sua saída), deixando a responsabilidade de perceber a saída de um nó para os demais, que devem se comunicar entre si periodicamente para perceber tais eventos. Uma vez notada a saída de um nó, os demais precisam ajustar as suas propriedades de objetos, trocando dados entre si.

Armazenamento de dados

Em se tratando da forma como os dados são armazenados, os bancos de dados NoSQL apresentam diferenças com relação aos bancos de dados relacionais. Essas diferenças referem-se à forma como os dados são armazenados e acessados em disco e aos modelos de consultas utilizados. Com relação ao primeiro aspecto, ele determina como os dados serão acessados em disco, afetando diretamente o desempenho, além de indicar que tipo de dados poderá ser lido em blocos. Os principais *layouts* para armazenamento são descritos a seguir (TIWARI, 2011).

- **Baseado em linhas:** nesse *layout* (Figura 2a), uma tabela é serializada com linhas inteiras, adicionadas ao fim da tabela, facilitando o acesso a conjuntos de dados em uma única operação de IO, pois possuem boa localidade de acesso a diferentes colunas.
- **Colunar:** ao contrário do *layout* baseado em linhas, no colunar (Figura 2b), as tabelas são serializadas com colunas, em vez de linhas, beneficiando operações sobre colunas, muito úteis em análise de dados, mas a custo de um desempenho sofrível em operações sobre linhas.

- **Colunar com grupos de localidade:** semelhante ao *layout* colunar, porém introduz os grupos de localidade, permitindo que as colunas sejam agrupadas quando se espera que elas sejam acessadas ao mesmo tempo (Figura 2c).
- **Árvores LSM (Log Sctructured Merge):** esse *layout* contrasta com todos os outros descritos, pois ele não procura serializar estruturas de dados lógicas, mas sim utilizar a memória e o disco de forma eficiente, servindo tanto requisições de leitura como escrita de forma eficiente, com alto desempenho e de forma segura. Os dados são armazenados em pedaços na memória (Memtables) e *commit-logs* são mantidos em disco para esses dados. De tempos em tempos, os dados em memória são descarregados em disco em SSTables.

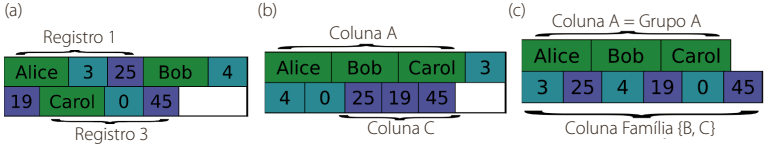


Figura 2. Diferenças entre *layouts* de armazenamento: (a) baseado em linhas; (b) colunar; (c) colunar com localidade de grupo.

Fonte: Adaptada de Strauch (2011).

Já com relação às funcionalidades de consulta em bancos de dados NoSQL, elas são muito variadas, partindo da premissa de simples *lookups* de chaves primárias, como é o caso dos bancos de dados de chave/valor. Além disso, existem algumas técnicas, como o *scatter/gather*, em que uma consulta é enviada para vários nós do banco de dados e, posteriormente, os resultados retornados são processados e entregues ao cliente. Um exemplo disso é o MapReduce, introduzido pela Google em 2004, em que pedaços de dados são designados a determinados nós que irão executar uma função `map` e devolver um resultado, o qual será processado por máquinas através da função `reduce` para criar o resultado final.



Saiba mais

O MapReduce é um modelo de processamento de dados muito simples, o qual deu início ao movimento *big data*. Hoje, existem diversos modelos e *frameworks* para processamento de grandes volumes de dados, como o Hadoop e o Spark.

3 Principais bancos de dados NoSQL

Ao longo dos anos, diversos bancos de dados surgiram no mercado e ganharam proeminência. A seguir, serão apresentados os bancos de dados NoSQL mais utilizados no mercado de acordo com o *site* db-engines.com.

DBM

Criado por Ken Thompson e lançado pela AT&T em 1979, o DBM foi um dos precursores dos bancos de dados de chave/valor. Ele consiste em uma biblioteca para armazenamento de dados com um número fixo de *buckets* e técnicas de *hashing* para a recuperação rápida de dados (BRACHMAN; NEUFELD, 1992). Ao longo dos anos, o DBM foi reescrito diversas vezes, sendo que, em 1991, foi reescrito pela então SleepyCat Software e chamado de Berkeley DB, hoje um produto da Oracle.

Os bancos de dados de chave/valor Tokyo Cabinet e Kyoto Cabinet são implementações inspiradas no DBM, bem como o WiredTiger, um banco de dados com armazenamento orientado tanto a colunas como a linhas e que, hoje, é a *engine* por baixo do MongoDB (FEDOROVA *et al*., 2018).

Neo4j

O Neo4j é um banco de dados orientado a grafos *open source* escrito em Java, com suporte a transações ACID, acessível através da linguagem CQL (Cypher Query Language; ou Linguagem de Consulta Contextual, em português). A comunicação com o banco de dados pode ser feita através do protocolo HTTP (HyperText Transfer Protocol; ou Protocolo de Transferência de Hipertexto, em português) ou do protocolo Bolt. Esse banco de dados possui uma versão *community* livre para uso e outra versão *entreprise* paga, que fornece funcionalidades como *backup* e alta disponibilidade.

O Neo4j é utilizado para sistemas de recomendações, aplicações da Internet das Coisas (IoT, Internet of Things) em análise de redes sociais, para traçar a árvore genealógica, para catálogo de produtos e até mesmo para buscas na internet.

Google BigTable

Desenvolvido pela Google em 2004, o BigTable é um banco de dados colunar com compressão e de alta *performance* (CHANG *et al.*, 2008). Ele depende do GFS para o armazenamento dos dados no formato de SSTables, além de utilizar o paradigma de programação Mapreduce, também de autoria da Google, para a geração e a modificação de dados armazenados no BigTable.

CouchDB

Banco de dados orientado a documentos criado em 2005 e escrito em Erlang. Os documentos são armazenados no formato JSON, e as consultas, na forma de Mapreduce, e consultas via API HTTP podem ser realizadas. O CouchDB tem suporte a propriedades ACID no nível de documento, consistência eventual e replicação de dados (ANDERSON; LEHNARDT; SLATER, 2010).

Amazon Dynamo

Criado pela Amazon em 2007, o Dynamo é um banco de dados de chave/valor altamente disponível. Ele abre mão da garantia de isolamento dentre as propriedades ACID para garantir uma alta disponibilidade. O Dynamo foi utilizado como base para vários serviços da AWS, como o Amazon S3 e o DynamoDB (DECANDIA *et al.*, 2007).

De acordo com Decandia *et al.* (2007), o Dynamo faz uso de *hashing* consistente para o particionamento de dados, permitindo a escalabilidade linear do *cluster*. Além disso, ele utiliza Vector Clocks para a reconciliação de atualizações durante a leitura, bem como uma técnica de antientropia com Merkle tree para sincronizar réplicas com divergências e um protocolo de filiação baseado no Gossip para evitar ter um componente centralizado para controlar os nós do *cluster*.

MongoDB

O MongoDB é um banco de dados orientado a documentos que utiliza o formato BSON (Binary JSON) para armazenar os documentos. Ele foi escrito em C++, possui suporte completo a índices, realiza o mapeamento de arquivos em memória (com escrita atrasada) e possui suporte a alta disponibilidade e escalabilidade através de replicação, *failover* e *auto-sharding* (HOWS; MEMBREY; PLUGGE, 2015).

O MongoDB é um dos bancos de dados NoSQL mais populares e vem sendo utilizado para substituir os bancos de dados relacionais em aplicações *web*. Além disso, ele é utilizado no gerenciamento de conteúdo semiestruturado, na análise de dados em tempo real, para *log* de dados em alta velocidade e para *cache* e alta disponibilidade de dados. O MongoDB possui, ainda, suporte para transações, garantindo a atomicidade de operações dentro de um documento (o suporte a transações, abrangendo múltiplos documentos, passou a ser suportado na versão 4.0). Entretanto, ele não é recomendado para aplicações altamente transacionais e problemas que necessitem de SQL.

Cassandra

O Cassandra é um banco de dados colunar e distribuído, descentralizado e *open source* capaz de gerenciar grandes volumes de dados com alta disponibilidade e sem SPoF. Ele é capaz de escalar de forma linear até mais de mil nós, com uma arquitetura *peer-to-peer*, suporte multi-*data center* e uma linguagem de consulta chamada de CQL, similar à linguagem SQL (CARPENTER; HEWITT, 2016).

Dentro dos conceitos do Cassandra, o *keyspace* é equivalente a um *database* em bancos de dados relacionais, uma *column family* é equivalente a uma tabela, uma *row key* é equivalente a uma chave primária e colunas e valores são iguais em ambos os paradigmas. De acordo com Carpenter e Hewitt (2016), no Cassandra, a busca de registros pela chave é muito rápida, e os dados são armazenados de forma ordenada, permitindo buscas por *ranges* de valores. Contudo, o número de colunas permitido no Cassandra é ilimitado, e os nomes de colunas podem chegar a 64 KB, ao passo que os seus valores podem chegar até 2 GB.

Redis

O Redis (REmote DIctionary Service) é um banco de dados de chave/valor *open source* escrito em C com armazenamento em memória com persistência em disco. Ele pode ser utilizado como banco de dados, camada de *cache* ou sistema de mensageria.

O Redis é um banco de dados compatível com sistemas POSIX (Linux, BSD, OS X), possui um servidor *single-threaded*, em que todas operações são atômicas, e a maioria dos seus comandos executa com complexidade O(1). Todos os dados são escritos em memória, o que garante uma leitura e escrita muito rápidas. Além disso, os dados podem ser salvos em disco. O Redis também tem suporte à replicação e à divisão de dados através de *sharding* (CARLSON, 2013). Essas características fazem do Redis um banco de dados muito popular, sendo utilizado por empresas como Hulu, Twitter, Pinterest, Snapchat, GitHub, Airbnb e Square, além do fato de possuir clientes para mais de 50 linguagens de programação.

Apache HBase e Hypertable

O Hbase é um banco de dados colunar baseado no Google Bigtable, escrito em Java e com dependência no HDFS (Hadoop Distributed File System). O Hypertable também é um banco de dados colunar baseado no Google Bigtable, mas, ao contrário do HBase, ele foi escrito em C++ e não é *open source* (KHETRAPAL; GANESH, 2008).

Apache Druid

O Apache Druid é um banco de dados colunar *open source* escrito para ter alto desempenho na escrita de grandes volumes de dados em tempo real, além de permitir a indexação de dados para consultas em tempo real (YANG *et al*., 2014).

Apache Kudu

O Apache Kudu é um banco de dados colunar *open source* compatível com o ambiente Hadoop. Ele foi desenvolvido para o processamento de dados analíticos, além de ter suporte para índices, permitindo o *lookup* aleatório de dados. O modelo de dados do Apache Kudu é muito similar ao de um banco

de dados relacional, entretanto, a sua arquitetura permite a escalabilidade dos dados (BARANOWSKI *et al*., 2019).

ScyllaDB

O ScyllaDB é um banco de dados colunar *open source* com compatibilidade com o Apache Cassandra por meio do suporte dos mesmos protocolos utilizados pelo Cassandra: Thrift e CQL. Em contrapartida, o ScyllaDB oferece um *throughput* maior e latências menores do que o Cassandra, utilizando o mesmo formato de arquivos (SSTable), mas sendo escrito em C++, em vez de Java (SUNEJA, 2019).

O Quadro 1, a seguir, apresenta os principais bancos de dados NoSQL disponíveis.

Quadro 1. Lista dos principais bancos de dados NoSQL

Banco de dados	Licença	Tipo	Ano de criação	Criado/ Mantido por
MultiValue (PICK)	Proprietário	Multivalor	1965	TRW
DBM	Proprietário	Chave/valor	1979	AT&T
Lotus Domino	Proprietário	Documentos	1989	Lotus
GDBM	*Open source*	Chave/valor	1990	GNU
BerkeleyDB	*Open source*	Chave/valor	1991	Sleepycat/Oracle
TDBM	*Open source*	Chave/valor	1992	
Caché	Proprietário	Multivalor	1997	InterSystems
Metakit	*Open source*	Documentos	1997	Jean-Claude Wippler
Neo4j	*Open source*	Grafos	2000	Neo4j
db4o	*Open source*	Objetos	2000	Actian
Memcached	*Open source*	Chave/valor	2003	Danga Interactive

(Continua)

(Continuação)

Quadro 1. Lista dos principais bancos de dados NoSQL

Banco de dados	Licença	Tipo	Ano de criação	Criado/ Mantido por
BigTable	Proprietário	Colunar	2004	Google
Infogrid	*Open source*	Grafos	2005	
CouchDB	*Open source*	Documentos	2005	Apache
Tokyo Cabinet	*Open source*	Chave/valor	2006	FAL Labs
Dynamo	Proprietário	Chave/valor	2007	Amazon
MongoDB	*Open source*	Documentos	2007	MongoDB Inc.
Cassandra	*Open source*	Colunar	2008	Facebook/ Apache
Project Voldemort	*Open source*	Chave/valor	2008	LinkedIn/ Microsoft
Dynomite	*Open source*	Chave/valor	2008	
Hypertable	*Open source*	Colunar	2008	Zevents Inc.
Accumulo	*Open source*	Colunar	2008	Apache
Terrastore	*Open source*	Documentos	2009	Sergio Bossa
Redis	*Open source*	Chave/valor	2009	Redis Labs
Riak	*Open source*	Chave/valor	2009	Basho Technologies
HBase	*Open source*	Colunar	2009	Apache
Vertexdb	*Open source*	Grafos	2009	
ClickHouse	*Open source*	Colunar	2009	Yandex
RethinkDB	*Open source*	Documentos	2009	
OrientDB	*Open source*	Multimodelo	2010	OrientDB Ltd.
Couchbase	*Open source*	Documentos	2010	Couchbase Inc.
CosmosDB	Proprietário	Documentos	2010	Microsoft
LDBM	*Open source*	Chave/valor	2011	

(Continua)

(Continuação)

Quadro 1. Lista dos principais bancos de dados NoSQL

Banco de dados	Licença	Tipo	Ano de criação	Criado/ Mantido por
ArangoDB	*Open source*	Multi-modelo	2011	ArangoDB GmbH
Druid	*Open source*	Colunar	2011	Apache
DynamoDB	Proprietário	Chave/valor, documentos	2012	Amazon
Founda-tionDB	*Open source*	Multimodelo	2013	Apple
Scylla	*Open source*	Colunar	2014	ScyllaDB Inc.
Kudu	*Open source*	Colunar	2016	Cloudera/ Apache
TerminusDB	*Open source*	Grafos	2018	
Neptune	Proprietário	Grafos	2018	Amazon
DocumentDB	Proprietário	Documentos	2019	Amazon



Referências

ANDERSON, J. C.; LEHNARDT, J.; SLATER, N. *CouchDB:* the definitive guide: time to relax. Sebastopol: O'Reilly, 2010.

BAILIS, P.; GHODSI, A. Eventual consistency today: limitations, extensions, and beyond. *ACM Queue*, New York, v. 11, n. 3, p. 20-32, 2013.

BARANOWSKI, Z. *et al.* A prototype for the evolution of ATLAS EventIndex based on Apache Kudu storage. *In:* INTERNATIONAL CONFERENCE ON COMPUTING IN HIGH ENERGY AND NUCLEAR PHYSICS, 23., 2018, Sofia, Bulgaria. *Anais eletrônicos*… Sofia: EPJ Web of Conferences, 2019.

BRACHMAN, B. J.; NEUFELD, G. W. TDBM: a DBM library with atomic transactions. *In:* USENIX SUMMER 1992 TECHNICAL CONFERENCE, 1992, San Antonio. *Anais eletrônicos*… San Antonio: USENIX, 1992. p. 63-75.

CARLSON, J. *Redis in action*. Shelter Island: Manning, 2013.

CARPENTER, J.; HEWITT, E. *Cassandra:* the definitive guide: distributed data at web scale. 2. ed. Sebastopol: O'Reilly, 2016.

CHANG, F. *et al*. Bigtable: a distributed storage system for structured data. *ACM Transactions on Computer Systems (TOCS)*, New York, v. 26, n. 2, p. 1-26, 2008.

CHAPPLE, M. How referential integrity ensures database consistency. *In:* LIFEWIRE. New York, 5 apr. 2020. Disponível em: https://www.lifewire.com/referential-integrity--definition-1019181. Acesso em: 27 jun. 2020.

CODD, E. F. A relational model of data for large shared data banks. *Communications of the ACM*, New York, v. 13, n. 6, p. 377-387, jun. 1970.

CORBETT, J. C. *et al*. Spanner: Google's globally distributed database. *ACM Transactions on Computer Systems (TOCS)*, New York, v. 31, n. 3, p. 1-22, 2013.

DECANDIA, G. *et al.* Dynamo: Amazon's highly available key-value store. *ACM SIGOPS operating systems review*, New York, v. 41, n. 6, p. 205-220, 2007.

FAOUR, N. Data consistency simulation tool for NoSQL database systems. *In:* arXiv. Ithaca, 2018. Disponível em: https://arxiv.org/abs/1802.08052. Acesso em: 27 jun. 2020.

FEDOROVA, A. *et al.* Performance comprehension at WiredTiger. *In:* ACM JOINT MEETING ON EUROPEAN SOFTWARE ENGINEERING CONFERENCE AND SYMPOSIUM ON THE FOUNDATIONS OF SOFTWARE ENGINEERING, 26., 2018, Lake Buena Vista. *Anais*… New York: Association for Computing Machinery, 2018. p. 83-94.

GILBERT, S.; LYNCH, N. Perspectives on the CAP theorem. *Computer*, New York, v. 45, n. 2, p. 30-36, 2012.

HAUGEN, K. A brief history of NoSQL. *In:* ALL about the code. *[S. l.]*, 16 mar. 2010. Disponível em: http://blog.knuthaugen.no/2010/03/a-brief-history-of-nosql.html. Acesso em: 27 jun. 2020.

HOWS, D.; MEMBREY, P.; PLUGGE, E. *Introdução ao MongoDB*. São Paulo: Novatec, 2015.

KHETRAPAL, A.; GANESH, V. HBase and Hypertable for large scale distributed storage systems. *[S. n.], [s. l.]*, 2008. Disponível em: https://pdfs.semanticscholar.org/38e2/6a2682f14663a54a75c8af9a6cd350d4c1c1.pdf?_ga=2.85776923.381094989.1593266258-971234948.1587415269. Acesso em: 27 jun. 2020.

KOKAY, M. C. Banco de dados NoSQL: um novo paradigma. *Revista SQL Magazine 102*, Rio de Janeiro, 2012. Disponível em: http://www.devmedia.com.br/banco-de-dados--nosql-um-novoparadigma-revista-sql-magazine-102/25918. Acesso em: 27 jun 2020;

KUZNETSOV, S. D.; POSKONIN, A. V. NoSQL data management systems. *Programming and Computer Software*, Moscow, v. 40, n. 6, p. 323-332, 2014.

LEAVITT, N. Will NoSQL databases live up to their promise? *Computer*, New York, v. 43, n. 2, p. 12-14, 2010.

PRAKASH, R.; SINGHAL, M. Dependency sequences and hierarchical clocks: efficient alternatives to vector clocks for mobile computing systems. *Wireless Networks*, Berlin, v. 3, n. 5, p. 349-360, 1997.

PRITCHETT, D. Base: an acid alternative. *ACM Queue*, New York, v. 6, n. 3, p. 48-55, 2008.

SADALAGE, P.; FOWLER, M. *NoSQL essencial:* um guia conciso para o mundo emergente da persistência. São Paulo: Novatec, 2019.

SHAPIRO, M. *et al.* Conflict-free replicated data types. *In:* DÉFAGO X.; PETIT, F.; VILLAIN, V. (Eds.). *Stabilization, safety, and security of distributed systems*: lecture notes in computer science, v. 6976. Berlin: Springer, 2011. p. 386-400.

STRAUCH, C. *NoSQL databases. [S. l.: s. n.]:* 2011. Disponível em: https://www.christof--strauch.de/nosqldbs.pdf. Acesso em: 27 jun. 2020.

SUNEJA, N. Scylladb optimizes database architecture to maximize hardware performance. *IEEE Software*, New York, v. 36, n. 4, p. 96-100, 2019.

TIWARI, S. *Professional NoSQL*. *[S. l.]:* Wrox, 2011.

YANG, F. *et al.* Druid: a real-time analytical data store. *In:* ACM SIGMOD INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA, 2014, Snowbird. *Anais*… New York: Association for Computing Machinery, 2014. p. 157-168.

ZVAREVASHE, K.; GOTORA, T. T. A random walk through the dark side of NoSQL databases in big data analytics. *International Journal of Science and Research*, Raipur, v. 3, n. 6, p. 506-509, 2014.

Leituras recomendadas

FOWLER, A. *NoSQL for Dummies*. New Jersey: Wiley, 2015.

PERKINS, L.; REDMOND, E.; WILSON, J. Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. 2. ed. *[S. l.]*: Pragmatic Bookshelf, 2018.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Dica do Professor: História dos bancos de dados não relacionais

Na medida em que o volume de dados processados e armazenados nas empresas foi crescendo, também cresceu a necessidade de bancos de dados capazes de comportar essa demanda. Foi aí que surgiu a necessidade de criar bancos de dados distribuídos, capazes de armazenar e processar quantidades enormes de dados. Entretanto, a distribuição dos dados também trouxe algumas limitações.

Na Dica do Professor, você conhecerá o Teorema de CAP e aprenderá quais são essas limitações e como elas influenciam a escolha do banco de dados mais adequado para cada situação.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Saiba mais: História dos bancos de dados não relacionais

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Introdução ao NoSQL

Assista neste vídeo uma introdução aos conceitos de bancos de dados NoSQL e suas características.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

History repeats itself: sensible and nonsenSQL aspects of the NoSQL hoopla (em inglês)

Este artigo da ACM conta a história dos bancos de dados NoSQL, o contexto no qual eles surgiram, as diferenças e semelhanças com relação aos bancos de dados relacionais e algumas críticas construtivas.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

A survey on NoSQL stores (em inglês)

Veja neste artigo do tipo survey uma descrição completa sobre a história e evolução dos bancos de dados NoSQL, suas características e decisões técnicas tomadas, além dos desafios que bancos de dados NoSQL têm para o futuro.

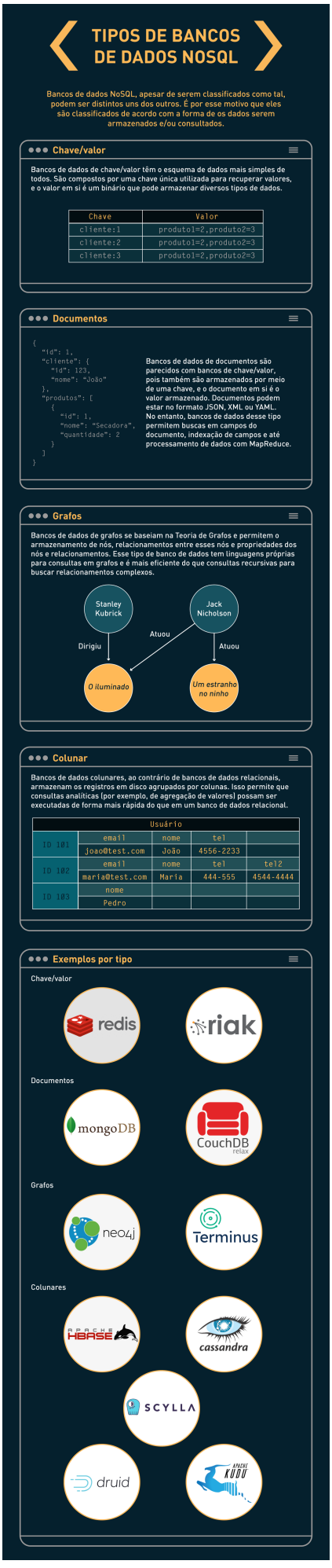


Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Infográfico: Classificação dos bancos de dados não relacionais

A palavra *NoSQL* se refere a uma classe de bancos de dados com características diversas, tanto no formato de dados quanto na forma de os dados serem armazenados. Conhecer os diferentes tipos de bancos de dados NoSQL é fundamental para saber escolher qual é a melhor opção para cada situação e tipo de problema. Além do formato dos dados e do modo como eles estão armazenados, é preciso conhecer os pontos fortes e as limitações de cada tipo de banco de dados NoSQL.

No Infográfico a seguir, veja os diversos tipos de bancos de dados NoSQL existentes, o formato de dados que cada um deles comporta, bem como exemplos de cada uma de suas categorias.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Conteúdo do Livro: Classificação dos bancos de dados não relacionais

Com volumes cada vez maiores de dados para serem armazenados e processados, tecnologias tradicionais de bancos de dados deixaram de ser adequadas, trazendo à tona uma nova onda de tecnologias atualmente conhecidas como NoSQL. Bancos de dados NoSQL são mais flexíveis e, em alguns casos, abrem mão de algumas garantias existentes em bancos de dados relacionais, em troca de um maior desempenho e uma maior escalabilidade.

Conhecer os diferentes tipos de bancos de dados NoSQL é imprescindível para saber como tirar melhor proveito dessas tecnologias e auxiliar negócios a crescerem e a comportarem demandas cada vez maiores para armazenamento e processamento de dados.

No capítulo Classificação dos bancos de dados não relacionais, da obra *Banco de dados não relacional*, base teórica desta Unidade de Aprendizagem, conheça os tipos de bancos de dados NoSQL existentes, suas características e seus representantes, bem como exemplos de aplicações e casos de uso de cada um deles. Além disso, aprenda os princípios BASE de consistência que são aplicados na maioria dos bancos de dados NoSQL.

Boa leitura.

BANCO DE DADOS NÃO RELACIONAL

Maycon Bordin



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Classificação dos bancos de dados não relacionais

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever os tipos de bancos de dados NoSQL.
- Discutir os princípios BASE dos bancos de dados NoSQL.
- Demonstrar a aplicabilidade dos diversos tipos de bancos de dados NoSQL.

Introdução

Em virtude do crescimento dos sistemas para a internet, foi necessária a implementação de soluções inovadoras para atender ao crescente volume de dados a ser analisado e armazenado. Lideradas por grandes empresas, essas inovações levaram à criação de uma série de bancos de dados dos mais diversos tipos e com variadas características, os quais, hoje, são conhecidos como bancos de dados não relacionais (NoSQL).

Neste capítulo, você conhecerá quais são os principais tipos de bancos de dados NoSQL, bem como quais são as suas principais características. Além disso, conhecerá o modelo de consistência de dados dos bancos de dados NoSQL, chamado de BASE, criado como alternativa às propriedades ACID dos bancos de dados relacionais. Por fim, verá onde esses bancos de dados podem ser aplicados e em que situações cada um deles é mais adequado.

1 Tipos de bancos de dados NoSQL

Apesar de o termo NoSQL ter surgido posteriormente ao surgimento de boa parte dos bancos de dados considerados não relacionais, eles foram agrupados nessa categoria por terem algumas semelhanças com relação ao problema que se propuseram a resolver: lidar com grandes volumes de dados através de escalabilidade horizontal, ser *open source* em sua maioria e lidar com tipos de dados diversos, muitas vezes sem um esquema de dados definido. A seguir, serão apresentados os principais tipos de bancos de dados NoSQL e suas características.

Bancos de dados de chave/valor

Os bancos de dados de chave/valor utilizam um modelo de dados bem simples, geralmente um *map* (ou HashMap) ou dicionário (ou *array* associativo), em que os clientes armazenam e requisitam valores através de chaves. Essas chaves são únicas e, com frequência, estão limitadas a uma certa quantidade de bytes. Esse tipo de banco de dados é muito eficiente (big O(1)) e favorece para a alta escalabilidade sobre a consistência, omitindo, na maior parte dos casos, funcionalidades muito elaboradas para consultas e análise (como *joins* e agregações).

Embora o termo NoSQL seja relativamente novo, os bancos de dados de chave/valor existem a um bom tempo, como o Berkeley DB. No segmento de sistemas de *cache*, aplicações como EHCache e memcached também seguem a ideia dos bancos de dados de chave/valor, apesar de não permitirem, em alguns casos, a persistência dos dados. Contudo, os bancos de dados que surgiram recentemente são inspirados em outro banco de dados, o Amazon Dynamo.

O Dynamo foi criado em um contexto em que muitos servidores estão distribuídos geograficamente, um *hardware* comum é utilizado, falhas em componentes são situações normais e a filosofia de SOA (Service-Oriented Architecture; ou Arquitetura Orientada a Serviços, em português) é fortemente aplicada (DECANDIA, 2007). Esse banco de dados é particionado através de *hashing* consistente, em conjunto com replicação. Os objetos nele armazenados, bem como alguns mecanismos, são versionados, para a garantia da consistência de dados durante atualizações e da detecção de falhas através de um protocolo baseado em *Gossip*, que permite adicionar e remover servidores com o mínimo de trabalho manual (DECANDIA, 2007).

Grande parte dos bancos de dados de chave/valor compartilham uma interface semelhante, que consiste, basicamente, de um método para salvar pares chave/valor (`set(key, value)` ou `put`), outro método para recuperá-los (`get(key)`) e um terceiro para removê-los (`delete(key)`). Além do Amazon Dynamo, existem outros bancos de dados de chave/valor, como: Project Voldemort, Tokyo Cabinet, Redis, MemcacheDB, Scalaris, Riak, ArangoDB e FoundationDB.

Bancos de dados orientados a documentos

Os bancos de dados orientados a documentos são considerados o próximo passo após os bancos de dados de chave/valor, pois permitem o armazenamento de informações mais complexas, com pares de chave/valor sendo armazenados na forma de documentos, e, em geral, adotam formatos como JSON (JavaScript Object Notation; ou Notação de Objetos JavaScript, em português), BSON (JSON binário), XML (eXtensible Markup Language; ou Linguagem de Marcação Extensível, em português) ou YAML (YAML Ain't Markup Language; ou YAML Não é Linguagem de Marcação, em português). Nesses bancos de dados, é possível ter documentos diferentes em uma mesma coleção, bem como criar índices não apenas para a chave, mas também para campos do documento.

Os bancos de dados orientados a documentos possuem interfaces para consulta, que permitem, inclusive, a aplicação de filtros, agregações, projeções e basicamente tudo o que pode ser feito em linguagens SQL (Structured Query Language; ou Linguagem de Consulta Estruturada, em português). Dentre as opções *open source* disponíveis, o MongoDB e o CouchDB são as mais proeminentes. Ambos permitem a replicação de dados, mas apenas o MongoDB fornece uma solução pensada para *sharding* de dados. Já com o CouchDB, a situação é semelhante ao MySQL. O MongoDB fornece, ainda, uma linguagem de consultas, feitas através de objetos BSON (semelhantes ao JSON), além de permitir a execução de funções JavaScript no servidor ou de MapReduce.

Além do MongoDB e do CouchDB, existem outros bancos de dados orientados a documentos, como: ArangoDB, Couchbase, CosmosDB, DocumentDB, OrientDB, RethinkDB e SimpleDB.

Bancos de dados orientados a grafos

Os bancos de dados orientados a grafos são baseados na teoria de grafos, com as entidades armazenadas na forma de nós (ou vértices), e os relacionamentos, na forma de arestas, com estes podendo guardar informações adicionais.

A Figura 1, a seguir, apresenta um exemplo de grafo, no qual os nós representam pessoas e lugares e os relacionamentos ligam as pessoas aos lugares que elas já visitaram. Além das informações que estão ilustradas na imagem, também seria possível armazenar dados adicionais sobre as pessoas, os lugares, bem como informações de quando a pessoa viajou para determinado lugar.

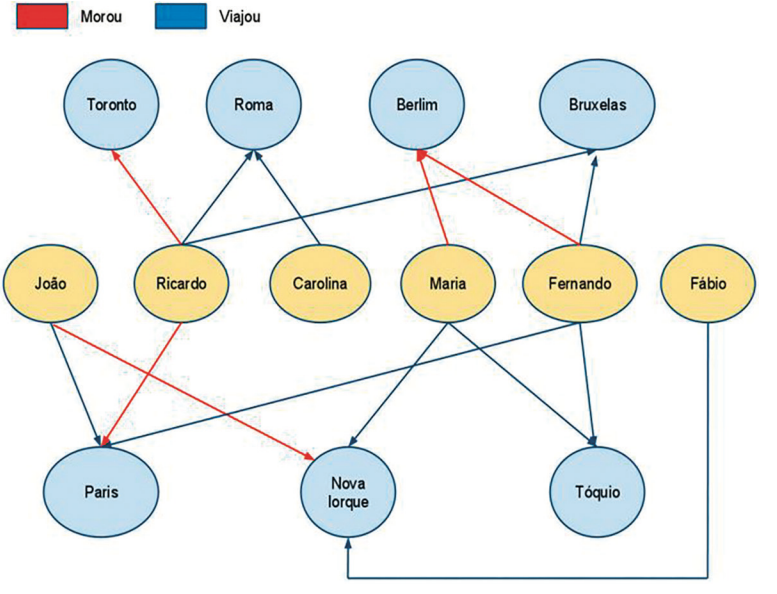


Figura 1. Exemplo de grafo.

Fonte: Kokay (2012, documento *on-line*).

A vantagem dos bancos de dados orientados a grafos está na busca rápida de conexões e relacionamentos entre objetos, devido ao fato de estes serem armazenados, em vez de calculados em tempo de execução. O mais popular entre os bancos de dados orientados a grafos é o Neo4j, o qual tem suporte a transações ACID e à linguagem de consultas CQL (Cypher Query Language; ou Linguagem de Consulta Contextual, em português).

Além do Neo4j, existem outros bancos de dados orientados a grafos, como: Amazon Neptune, ArangoDB, JanusGraph, OrientDB e TerminusDB.

Bancos de dados orientados a colunas

Os bancos de dados orientados a colunas armazenam os dados por colunas, em vez de por linhas, como nos bancos de dados relacionais. Essa abordagem surgiu na análise de dados e *business intelligence* (BI) com o uso de armazenamento em colunas em arquiteturas *shared-nothing*, a fim de criar uma arquitetura de processamento altamente paralela para aplicações de alto desempenho.

Entretanto, os bancos de dados orientados a colunas são menos puristas do que os produtos pensados para o BI, como Sybase IQ e Vertica, e alguns, inclusive, misturam orientação à colunas com orientação a linhas. O exemplo mais notável desse tipo de banco de dados é o Google Bigtable, que serviu, posteriormente, como inspiração para outros bancos de dados do mesmo tipo, como o Hypertable e o HBase.

Além dos bancos de dados orientados a colunas citados, destacam-se: Apache Cassandra, Apache Accumulo, Apache Kudu, ClickHouse, Druid e Scylla.



Fique atento

A maioria dos autores que estudam bancos de dados NoSQL abordam apenas os quatro principais tipos: chave/valor, documentos, grafos e colunares. Entretanto, existem autores que fazem menção a outros tipos de bancos de dados que não se encaixam como relacionais e poderiam ser considerados NoSQL, como: bancos de dados de tuplas/triplas/RDF, bancos de dados multivalor e bancos de dados orientados a objetos.

2 Princípios BASE dos bancos de dados NoSQL

Com o crescimento da internet, ter uma aplicação popular significa manter um grande número de transações. No entanto, se essa aplicação se utiliza de persistência de dados, esse pode ser um possível gargalo. Para garantir que as aplicações consigam atender a um número cada vez maior de transações, é possível escalar essa aplicação verticalmente, ao trocar o *hardware* atual por um *hardware* melhor, ou horizontalmente, adicionando mais máquinas e dividindo, de alguma forma, os dados e o processamento (PRITCHETT, 2008).

Contudo, a escalabilidade vertical tem suas limitações, pois, em determinado momento, não haverá mais *hardware* melhor para ser utilizado. Além disso, ela muitas vezes implica o *lock-in* em um único fornecedor, dificultando a migração para outro banco de dados no futuro. Assim, a alternativa é a escalabilidade horizontal, seja dividindo a aplicação em áreas funcionais, seja dividindo os dados entre as máquinas de um *cluster*.

Todavia, escalar um banco de dados horizontalmente implica, de modo invariável, abrir mão de algumas garantias. Para quem está habituado com bancos de dados relacionais, essas garantias se traduzem no acrônimo ACID, que significa (PRITCHETT, 2008):

- **Atomicidade:** todas as operações na transação serão completadas, ou nenhuma delas será.
- **Consistência:** o banco de dados ficará em um estado válido no início e ao fim da transação.
- **Isolamento:** a transação se comportará como se fosse a única transação executada no banco de dados.
- **Durabilidade:** após completar a transação, a operação não será revertida.

Ao adicionar o particionamento dos dados (i.e., divisão em mais de uma máquina), faz-se necessário, de alguma forma, garantir que as transações serão aplicadas em todas as máquinas. Para resolver esse problema, surgiu uma técnica chamada de 2PC (Two Phase Commit; ou Commit de Duas Fases, em português). Esse protocolo, como o nome sugere, trabalha em duas fases. Na primeira fase, ele pergunta para todos os bancos de dados envolvidos se eles podem aplicar determinado *commit*. Já na segunda fase, o *commit* é, de fato, aplicado em cada um dos bancos de dados. Uma desvantagem dessa técnica é que a disponibilidade do sistema é produto da disponibilidade individual de cada um dos bancos de dados envolvidos, afetando, assim, a disponibilidade do sistema como um todo.

Para garantir a disponibilidade em um sistema particionado, foram criados os princípios BASE, como uma alternativa aos princípios ACID (GC, 2016). As propriedades BASE relaxam as propriedades de consistência e isolamento em troca de disponibilidade, *graceful degradation* e desempenho. BASE significa *basic available*, *soft-state*, *eventual consistency*, que pode ser traduzido em uma aplicação que está sempre funcionando, nem sempre está consistente, mas eventualmente estará em um estado conhecido.

Quando um sistema exige consistência, no sentido de que toda a leitura precisa retornar, sem exceções, os dados da última operação de escrita, independentemente do nó onde as operações ocorreram, isso significa que ambas as operações (leitura e escrita) precisam ocorrer no mesmo nó ou deve haver algum mecanismo (p. ex., *commit* de duas fases e protocolos de validação de *cache*) para garantir tal propriedade. Já a consistência eventual relaxa esses requisitos, afirmando que, em algum momento, as operações de escrita aparecerão para os clientes que estão fazendo a leitura. O Quadro 1, a seguir, apresenta as principais diferenças entre as propriedades ACID e BASE.

Quadro 1. Lista das principais diferenças entre propriedades ACID e BASE

ACID	BASE
Consistência forte	Consistência fraca
Isolamento	Foco em disponibilidade
Concentra-se em *commit*	Melhor esforço
Transações aninhadas	Respostas aproximadas
Disponibilidade	Mais simples e mais rápido
Conservador (pessimista)	Agressivo (otimista)
Evolução difícil	Evolução mais fácil

Fonte: Adaptado de Brewer (2000).

Ao contrário das propriedades ACID, que são pessimistas e forçam a consistência ao final de cada operação, as propriedades BASE são otimistas e aceitam que a consistência do banco de dados não será constante. A disponibilidade é atingida por sistemas BASE através do suporte parcial a falhas, ou seja, sem ocasionar uma falha completa do sistema.

A Figura 2, a seguir, ilustra as diferenças entre os bancos de dados relacionais que implementam as propriedades ACID e fazem uso do protocolo 2PC os bancos de dados NoSQL que implementam propriedades BASE. A ilustração deixa claro por que os bancos de dados NoSQL conseguem escalar muito mais do que os bancos de dados relacionais.

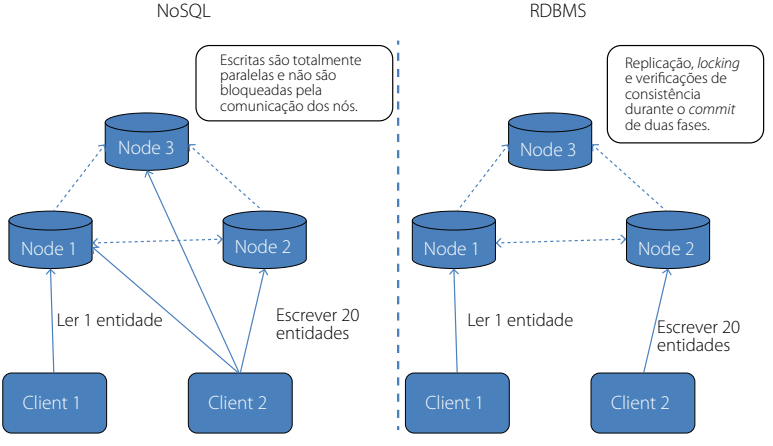


Figura 2. Diferença entre escrita e leitura em um banco de dados relacional (ACID) e um banco de dados NoSQL (BASE).

Fonte: Adaptada de Pritchett (2008).

Ao analisar mais de perto as diferenças entre ACID e BASE, é possível perceber que essas propriedades são, de certa forma, opostas. Ambas as propriedades podem ser explicadas por meio do teorema CAP (*consistency*, *availability*, *partition tolerance*), desenvolvido pelo professor Eric Brewer, da Universidade da Califórnia, em Berkeley. Ele desenvolveu uma conjectura de que os sistemas em rede que compartilham dados podem ter, ao mesmo tempo, apenas duas das propriedades listadas a seguir.

- **Consistência (*consistency*):** indica se e quando um sistema está em um estado consistente após a execução de uma operação. Por exemplo, se uma atualização é feita em um escritor do sistema e todos os leitores conseguem ver essa atualização imediatamente, o sistema é consistente.
- **Disponibilidade (*availability*):** o sistema deve permanecer disponível mesmo após a falha de nós ou a indisponibilidade de algum *hardware* ou *software*.
- **Partição tolerante a falhas (*partition tolerance*):** habilidade de um sistema de se manter operante mesmo com partições na rede, isolando alguns nós em ilhas. Pode significar, ainda, a capacidade de um sistema de lidar com a adição ou a remoção de nós.

Dentro do teorema de CAP, os sistemas CA possuem propriedades ACID e têm forte consistência, porém não suportam falhas de partição. Já os sistemas AP possuem propriedades BASE, sendo sistemas com alta disponibilidade e tolerantes a falhas de partição. Existem, ainda, os sistemas CP, que possuem consistência forte e são tolerantes a falhas de partição (BREWER, 2012).

É importante ressaltar que outros teoremas surgiram para complementar o teorema CAP, como é o caso do teorema de PACELC. Esse teorema afirma que, no caso de uma partição de rede (P) em um sistema distribuído, é preciso escolher entre disponibilidade (A) e consistência (C) (assim como no teorema CAP). No entanto, mesmo se o sistema estiver funcionando corretamente (E), será preciso escolher entre latência (L) e consistência (C) (GOLAB, 2018).



Saiba mais

Em sistemas distribuídos, o assunto consistência possui vasta literatura e estudos relacionados. O próprio teorema CAP é criticado por alguns cientistas da computação que propuseram alternativas ou, ao menos, cautela no seu uso, como é o caso de Kleppmann (2015), que propõe um *framework* para avaliar a consistência de sistemas distribuídos em redes que possam apresentar falhas. O próprio autor do teorema CAP (BREWER, 2012) faz uma reflexão sobre o seu trabalho, mostrando que esse modelo é mais flexível do que parece.

3 Aplicabilidade dos diversos tipos de bancos de dados NoSQL

Os bancos de dados NoSQL surgiram para resolver problemas que não podiam ser solucionados com os bancos de dados relacionais, principalmente com relação à escalabilidade desses sistemas, algo cada vez mais necessário em virtude do crescimento contínuo do volume de dados a ser armazenado e processado.

Todavia, os bancos de dados NoSQL também possuem características que podem fazer deles mais adequados para determinadas situações e tipos de aplicação. A seguir, será apresentada a aplicabilidade de cada um dos tipos de bancos de dados NoSQL.

Chave/valor

Segundo Knight (2017), os bancos de dados de chave/valor são ideais para: dados que escalam com facilidade; armazenamento de perfis, dados de preferências, configurações e dados de sessões, que podem ser acessados através da ID do usuário; gerenciamento de *cache* de dados; implementação de *blockchain*; e armazenamento de multimídia ou de objetos grandes.

De acordo com a Hazelcast, os bancos de dados de chave/valor também são utilizados em sistemas de recomendação em tempo real e para anúncios, pois permitem um rápido acesso aos dados para exibir recomendações e anúncios para os visitantes de um *site* (WHAT IS..., 2020). Um exemplo de uso real desse tipo de banco é o Twitter, que utiliza o Redis com um *cluster* com mais de 3K nós, 10 TB de dados e volume superior a 10 milhões de consultas por segundo. No Twitter, foi criado um serviço chamado Nighthawk, que é um sistema de *cache* como serviço, permitindo que diversos times utilizem o Redis para os mais diversos casos de uso, como: serviços de *analytics* para anúncios e vídeo; fornecimento de negociação de anúncios; mensagens diretas; e *tracking* de conversas em dispositivos móveis (RAMESH, 2017).

No caso do Twitter, além de servir o Redis como serviço, ele precisou ser fornecido com escalabilidade vertical e horizontal, além de ter elasticidade para adicionar e remover nós, alto *throughput*, baixa latência e alta disponibilidade mesmo no caso de falhas de nós.

Documentos

Os bancos de dados orientados a documentos são sistemas muito úteis em tarefas de integração e migração de dados, uma vez que não forçam um esquema de dados. Segundo Hecht e Jablonski (2011), alguns casos populares desses tipos de bancos são *analytics* em tempo real, *logging* e camada de armazenamento de *sites* pequenos, como *blogs*.

De acordo com a Amazon, por terem um esquema de dados flexível, os bancos de dados orientados a documentos são ótimas opções para dados que tendem a variar com relação ao esquema, podendo ter mais ou menos campos, como é o caso de perfis de usuários, que podem ter diversos campos customizados de dados, uma vez que os bancos de dados orientados a documentos conseguem comportar essas variações com tranquilidade (DOCUMENT..., 2020). Além disso, eles podem ser utilizados para agregação de dados de diversas fontes em tempo real, bem como para processamento e uso em ferramentas de BI. Os bancos de dados orientados a documentos podem ser utilizados, ainda, para o gerenciamento de conteúdo (CMS, Content Management System), permitindo a agregação dos mais diversos tipos de conteúdos com os mais variados dados, o que é fácil de realizar nesse tipo de banco, devido à sua flexibilidade com relação ao esquema (DOCUMENT..., 2020).

Além dos casos de uso citados, o próprio *site* do MongoDB lista alguns exemplos de cenários em que um banco de dados de documentos pode ser utilizado, como: para visão única de dados diversos, tirando vantagem do esquema flexível de um banco de dados orientado a documentos; para sistemas de Internet das Coisas (IoT, Internet of Things), pois permite a ingestão de grandes volumes de dados e a análise destes em tempo real, além de permitir uma arquitetura orientada a eventos; para desenvolvimento de aplicações móveis; para personalização de conteúdo; para catálogos de produtos e bens; para mover cargas de trabalho para fora de *mainframes*; e para desenvolvimento de *back-ends* para jogos (USE CASES, 2020).

Colunas

Segundo Hecht e Jablonski (2011), os bancos de dados orientados a colunas são ideais para casos em que existem enormes quantidades de dados com um número variável de atributos. Talvez um dos principais casos de uso de banco de dados orientados a colunas seja o da Netflix com o Apache Cassandra, que, apesar de ter sido criado e utilizado pelo Facebook, hoje é muito utilizado

pelo serviço de *streaming*. O Cassandra é utilizado na Netflix para armazenar informações sobre mídias (imagens, vídeos, áudios, legendas), além de todo o histórico de visualização dos usuários (TIWARI *et al*., 2018).

O histórico de visualizações da Netflix é armazenado no Cassandra, que é utilizado como um banco de dados de *timeseries* e consultado por diversos times para compreender o comportamento dos usuários e melhorar a experiência com a Netflix. São mais de 100 milhões de usuários ao redor do mundo e mais de 140 milhões de horas de conteúdo assistidos diariamente na plataforma. Assim, o time de engenharia da Netflix precisou remodelar o esquema de dados no Cassandra, adicionar compactação de dados, além de adicionar uma camada de *cache* de dados. Com isso, eles conseguiram reduzir o uso de espaço pelos dados, além de melhorar tanto a escrita como a leitura de dados (DUVEDI *et al*., 2018).

Além do Cassandra, a Netflix também faz uso do Apache Druid, outro banco de dados orientado a colunas, utilizado para ingerir grandes volumes de dados com estatísticas de uso sobre os dispositivos dos usuários, com foco na qualidade dos serviços. Além disso, ele facilita a visualização da experiência de uso por grupos de dispositivos, o que possibilita a detecção de problemas que possam estar afetando grupos específicos de usuários. Os dados são particionados por *timestamp*, o que permite que o banco de dados escale para trilhões de linhas e, ainda assim, consiga responder a consultas em milissegundos. Na Netflix, o Druid ingere, atualmente, mais de 2 milhões de eventos por segundo, e consultas são feitos em cima de mais de 1,5 trilhão de linhas de dados (SYKES, 2020).

Outro caso de uso de um banco de dados orientado a colunas é o da Globo.com, que utiliza o Cassandra para o seu serviço de *streaming* de vídeo (Globosat Play e Globoplay), especificamente armazenando dados binários de vídeo no Cassandra e servindo seus usuários diretamente do banco de dados (MARTINS, 2016).

Saindo da área de *streaming* de vídeo, outro caso de uso interessante do Cassandra é o da Uber, que o utilizou no centro de sua nova plataforma de *machine learning*, chamada Michelangelo, utilizada pelo UberEATS para predizer o tempo de entrega de encomendas, disponibilizar o *ranking* de buscas e de restaurantes e autocompletar as buscas. Portanto, o Cassandra é utilizado especificamente para o acesso a *features* dos modelos de *machine learning* para predições em tempo real (HERMANN; BALSO, 2017).

Grafos

Os bancos de dados orientados a grafos são recomendados para aplicações com dados com muitos relacionamentos, pois, nesses casos, operações intensivas em processamento, como *joins* recursivos, podem ser substituídas por travessias eficientes (HECHT; JABLONSKI, 2011).

Um caso de uso muito palpável de bancos de dados orientados a grafos são as redes sociais, como é o caso do Twitter. Nesse caso específico, o Twitter criou o seu próprio banco de dados orientado a grafos, chamado FlockDB, para armazenar os relacionamentos entre os usuários da rede social. O FlockDB é capaz de atender a picos de milhões de consultas por segundo (QPS, Queries Per Second) e uma média de 30 a 40 k QPS (HASHEMI, 2017).

De acordo com Chapel (2019), os bancos de dados orientados a grafos podem ser utilizados para: aplicações de *machine learning*, como sistemas de reconhecimento de imagens e fala, *chatbots* e motores de recomendação; detecção de fraudes, permitindo trabalhar com grandes volumes de dados transacionais para detecção de fraudes; conformidade regulatória, como as leis HIPAA (Health Insurance Portability and Accountability Act; ou Lei de Responsabilidade e Portabilidade de Seguro Saúde, em português) e GDPR (General Data Protection Regulation; ou Regulamento Geral de Proteção de Dados, em português), similar à LGPD (Lei Geral de Proteção de Dados); grafos de conhecimento, permitindo buscas por palavras-chave e buscas complexas por conteúdo, modelos de doenças e interações de genes, padrões de proteínas e componentes químicos; e modelagem de cadeia de suprimentos complexos.



Referências

BREWER, E. A. CAP twelve years later: How the "rules" have changed. *Computer*, Washington, v. 45, n. 2, p. 23–29, 2012. Disponível em: https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/. Acesso em: 7 jul. 2020.

BREWER, E. A. Towards Robust Distributed Systems. *In*: ACM Symposium on Principles of Distributed Computing, 2000, Portland. *Lectures* [...]. Portland: ACM, 2000. 45 p. (Apresentação de slides). Disponível em: http://www.cs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf. Acesso em: 7 jul. 2020.

CHAPEL, J. AWS Neptune Overview — Amazon's Graph Database Service. *Faun*, [*S. l.*], 2 Sep. 2019. Disponível em: https://medium.com/faun/aws-neptune-overview-amazons-graph-database-service-9e3a0f688e26. Acesso em: 7 jul. 2020.

DECANDIA, G. *et al*. Dynamo: amazon's highly available key-value store. *ACM SIGOPS operating systems review*, [*S. l.*], v. 41, n. 6, p. 205–220, 2007.

DOCUMENT Database Use Cases. *Amazon Web Services*, Seattle, 2020. Disponível em: https://docs.aws.amazon.com/documentdb/latest/developerguide/document-database-use-cases.html. Acesso em: 7 jul. 2020.

DUVEDI, K. *et al*. Scaling Time Series Data Storage – Part I. *The Netflix Tech Blog*, Los Gatos, 23 Jan. 2018. Disponível em: https://netflixtechblog.com/scaling-time-series-data-storage-part-i-ec2b6d44ba39. Acesso em: 7 jul. 2020.

GC, D. *A critical comparison of NoSQL databases in the context of acid and base*. 2016. 71 f. Dissertação (Mestrado em Information Assurance) – Department of Information Assurance and Information Systems, St Cloud State University, St. Cloud, 2016. Disponível em: https://repository.stcloudstate.edu/msia_etds/8/. Acesso em: 7 jul. 2020.

GOLAB, W. Proving PACELC. *ACM SIGACT News*, Washington, v. 49, n. 1, p. 73–81, Mar. 2018.

HASHEMI, M. The Infrastructure Behind Twitter: Scale. *Twitter Engineering*, San Francisco, 19 Jan. 2017. Disponível em: https://blog.twitter.com/engineering/en_us/topics/infrastructure/2017/the-infrastructure-behind-twitter-scale.html. Acesso em: 7 jul. 2020.

HECHT, R.; JABLONSKI, S. NoSQL evaluation: A use case oriented survey. *In*: INTERNATIONAL CONFERENCE ON CLOUD AND SERVICE COMPUTING, 2011, Hong Kong. *Proceedings* [. . .]. Hong Kong: IEEE, 2011. p. 336–341.

HERMANN, J.; BALSO, M. D. Meet Michelangelo: Uber's Machine Learning Platform. *Uber Engineering*, San Francisco, 5 Sep. 2017. Disponível em: https://eng.uber.com/michelangelo-machine-learning-platform/. Acesso em: 7 jul. 2020.

KLEPPMANN, M. A Critique of the CAP Theorem. *arXiv*, [*S. l.*], 2015. Disponível em: https://arxiv.org/abs/1509.05393. Acesso em: 7 jul. 2020

KNIGHT, M. What is a Key-Value Database? *Dataversity*, Studio City, 30 Sep. 2017. Disponível em: https://www.dataversity.net/key-value-database/. Acesso em: 7 jul. 2020.

KOKAY, M. Banco de dados NoSQL: Um novo paradigma. *SQL Magazine*, Rio de Janeiro, n. 102, 2012. Disponível em: https://www.devmedia.com.br/banco-de-dados-nosql-um-novo-paradigma-revista-sql-magazine-102/25918. Acesso em: 7 jul. 2020.

MARTINS, A. Cassandra at the heart of Globo's live streaming platform. *Java Code Geeks*, Kesariani, 22 June 2016. Disponível em: https://www.javacodegeeks.com/2016/06/cassandra-heart-globos-live-streaming-platform.html. Acesso em: 7 jul. 2020.

PRITCHETT, D. Base: An Acid Alternative. *ACM Queue*, New York, v. 6, n. 3, p. 48–55, May/June 2008. Disponível em: https://queue.acm.org/detail.cfm?id=1394128. Acesso em: 7 jul. 2020.

RAMESH, R. *Nighthawk*: Distributed caching with Redis @ Twitter. San Francisco: RedisConf, 2017. 26 p. (Palestra na RedisConf17).

SYKES, B. How Netflix uses Druid for Real-time Insights to Ensure a High-Quality Experience. *The Netflix Tech Blog*, Los Gatos, 3 Mar. 2020. Disponível em: https://netflixtechblog.com/how-netflix-uses-druid-for-real-time-insights-to-ensure-a-high-quality-experience-19e1e8568d06. Acesso em: 7 jul. 2020.

TIWARI, S. *et al*. Implementing the Netflix Media Database. *The Netflix Tech Blog*, Los Gatos, 14 Dec. 2018. Disponível em: https://netflixtechblog.com/implementing-the-netflix-media-database-53b5a840b42a. Acesso em: 7 jul. 2020.

USE CASES. *MongoDB*, New York, 2020. Disponível em: https://www.mongodb.com/use-cases. Acesso em: 7 jul. 2020.

VALE, V. A. NoSQL. *Sudoers*, São Paulo, 17 fev. 2015. Disponível em: http://blog.sudoers.com.br/nosql/. Acesso em: 7 jul. 2020.

WHAT IS a key-value store? *Hazelcast*, San Mateo, 2020. Disponível em: https://hazelcast.com/glossary/key-value-store/. Acesso em: 7 jul. 2020.

Leituras recomendadas

FOWLER, A. *NoSQL for dummies*. Hoboken: Wiley, 2015. 456 p.

MOHAMED, M. A.; ALTRAFI, O. G.; ISMAIL, M. O. Relational vs. NoSQL databases: A survey. *International Journal of Computer and Information Technology*, [*S. l.*], v. 3, n. 3, p. 598–601, 2014. Disponível em: https://www.ijcit.com/archives/volume3/issue3/Paper030320.pdf. Acesso em: 7 jul. 2020.

PERKINS, L.; REDMOND, E.; WILSON, J. *Seven databases in seven weeks*: a guide to modern databases and the NoSQL movement. 2. ed. Pragmatic Bookshelf, 2018. 358 p.

SULLIVAN, D. *NoSQL for mere mortals*. Boston: Addison-Wesley Professional, 2015. 552 p.

TUDORICA, B. G.; BUCUR, C. A comparison between several NoSQL databases with comments and notes. *In*: ROEDUNET INTERNATIONAL CONFERENCE, 10., 2011, Iasi. *Proceedings* [...]. Iasi: IEEE, 2011. p. 1–5.



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Dica do Professor: Classificação dos bancos de dados não relacionais

Bancos de dados são muito importantes, pois são eles que armazenam o que há de mais valioso nas empresas. A fim de tudo estar seguro e disponível para acesso, bancos de dados precisam implementar diversos mecanismos para garantir que isso aconteça.

Nesta Dica do Professor, conheça as propriedades BASE que são aplicadas na maioria dos bancos de dados NoSQL e como elas se comparam com as propriedades ACID dos bancos de dados relacionais.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Saiba mais: Classificação dos bancos de dados não relacionais

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

NoSQL do zero (bancos de dados não relacionais)

Este vídeo contém uma introdução aos tipos de bancos de dados NoSQL e suas características.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Critérios para seleção de SGBD NoSQL: o ponto de vista de especialistas com base na literatura

Este artigo apresenta uma lista de requisitos que devem ser levados em conta na hora de escolher um banco de dados NoSQL, de acordo com especialistas da área.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Persisting big-data: the NoSQL landscape

Este artigo, em inglês, apresenta um estudo completo sobre os tipos de bancos de dados NoSQL e suas implementações, levantando as características e os funcionamentos de cada tipo de banco de dados NoSQL, comparando-os com bancos de dados relacionais. Confira.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Cassandra Serving Netflix @ Scale - Vinay Chella, Netflix

Acompanhe, neste vídeo, como a Netflix utiliza o Cassandra para armazenar seus dados em larga escala e com alto desempenho. O áudio está em inglês.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Mapeamentos conceituais entre os modelos relacional e NoSQL: uma abordagem comparativa

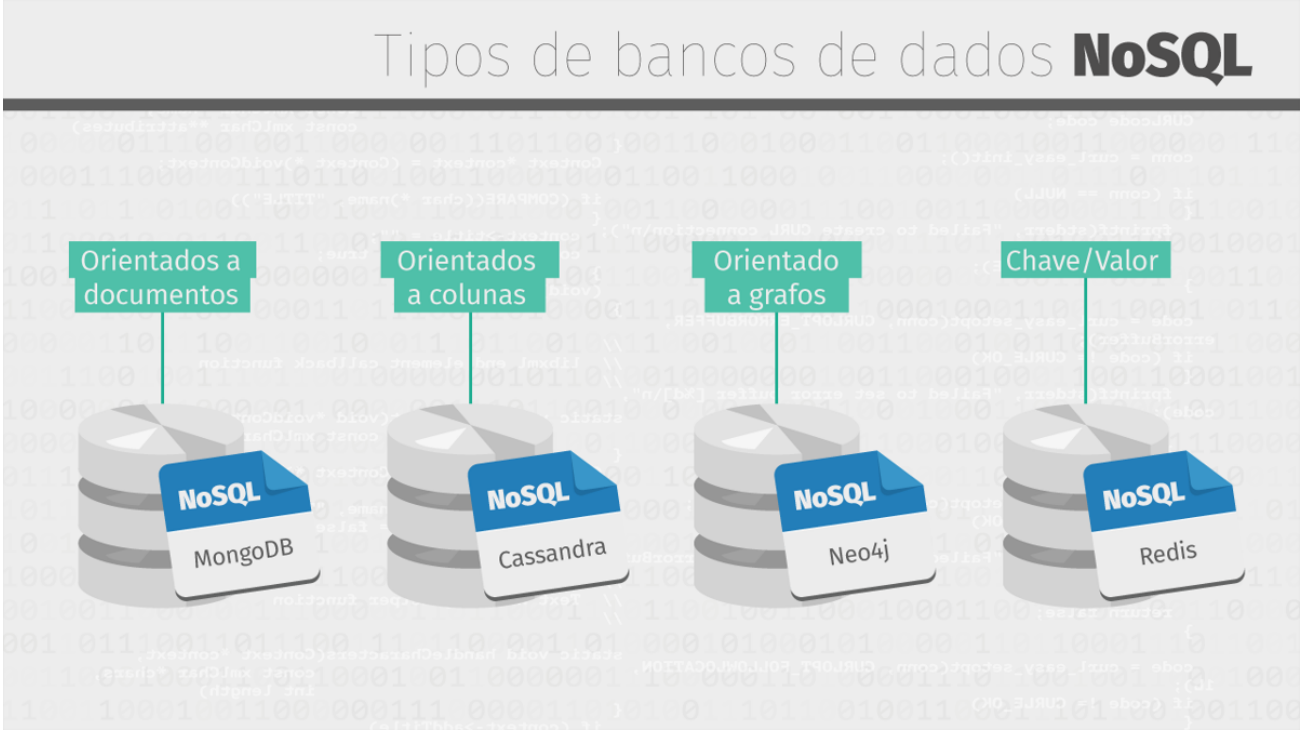
Este artigo apresenta um comparativo entre o modelo relacional e os diferentes tipos de bancos de dados NoSQL. Isso é feito por meio de um exemplo prático com a modelagem de dados em cada tipo de banco de dados.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Infográfico: Analisar e compreender a utilização do Banco de Dados NoSQL

Observe, no infográfico, um esquema que representa os tipos de bancos de dados NoSQL e um exemplo de banco de dados para cada modelo.

A imagem a seguir possui audiodescrição.



Conteúdo do Livro: Analisar e compreender a utilização do Banco de Dados NoSQL

No capítulo Analisar e compreender a utilização do banco de dados NoSQL, da obra *Administração de Banco de Dados*, você verá a diferença entre dados, informação e conhecimento, e a relação desses conceitos com o banco de dados. Além disso, aprenderá sobre Big Data e verá as principais características dos bancos de dados NoSQL.

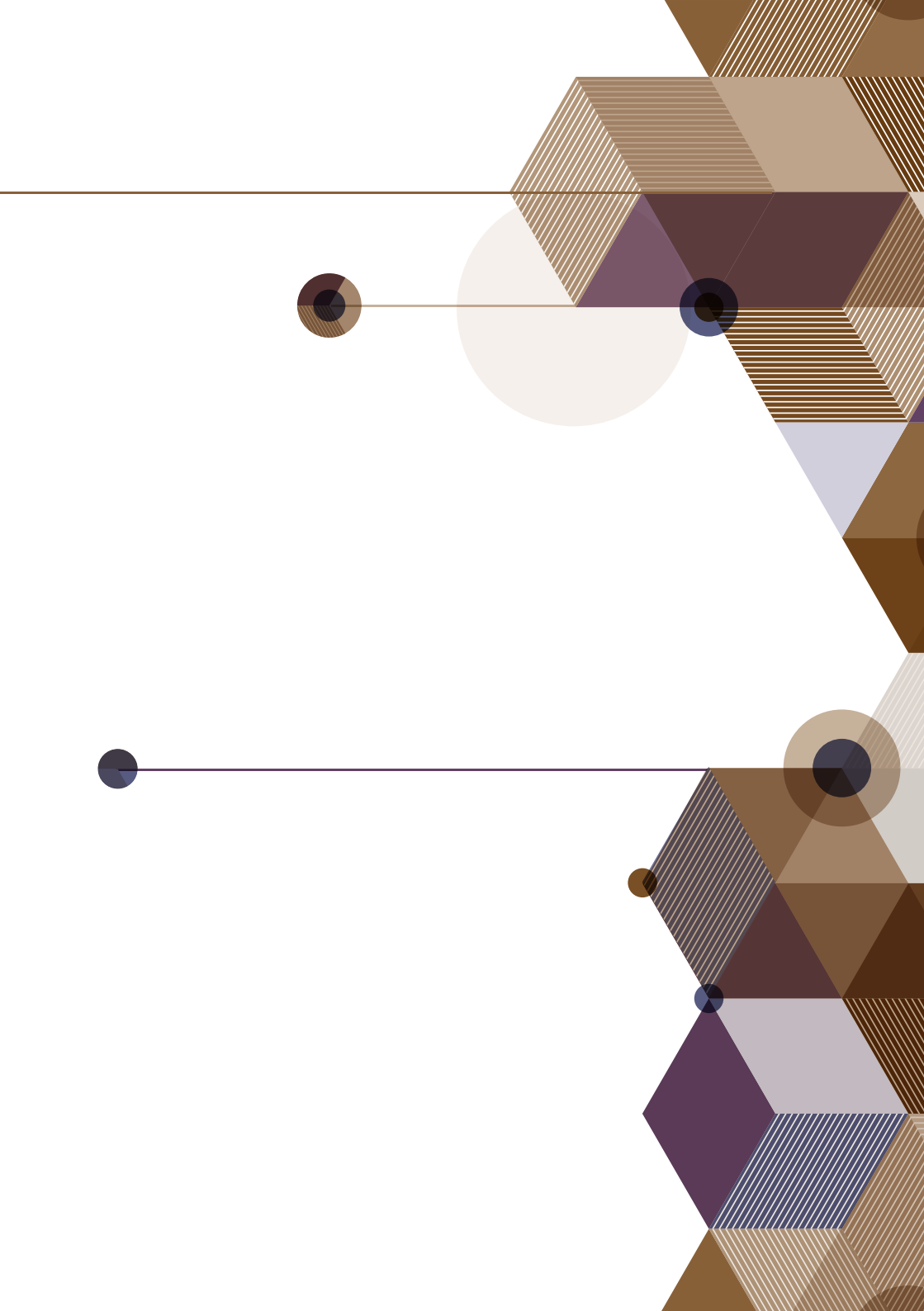
Boa leitura.

Os elementos gráficos deste capítulo possuem audiodescrição. Para acessar o recurso, clique aqui

ADMINISTRAÇÃO DE BANCO DE DADOS



Jorge Brandão de Campos



ANALISAR E COMPREENDER A UTILIZAÇÃO DO BANCO DE DADOS NoSQL

OBJETIVOS DE APRENDIZAGEM

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever o que são bancos de dados NoSQL.
- Listar os principais tipos e características dos bancos de dados NoSQL.
- Diferenciar o modelo relacional (SQL) e o NoSQL.

INTRODUÇÃO

O volume de dados está aumentando em proporções exponenciais no mundo. Os Estados Unidos é o país em que mais circulam dados, ficando a China em segundo lugar. Também merece destaque em tráfego de dados a América Latina, o Oriente Médio e a África.

E como lidar com tanta informação? Os bancos de dados de modelos relacionais estão preparados?

Neste texto, você vai estudar um novo conceito de banco de dados para trabalhar com grandes volumes de dados, o NoSQL.

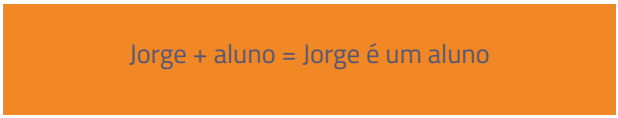
BANCOS DE DADOS

Antes de falarmos em bancos de dados, devemos conhecer alguns conceitos importantes. O que são dados, informação e conhecimento?

Podemos conceituar dados como sendo a forma bruta da informação, ou a parte indivisível da informação. Vejamos um exemplo: abra um bloco de notas e insira seu nome, "Jorge". Neste instante, façolhe uma pergunta: o que significa "Jorge"? A resposta é simples, é apenas um dado, não tem significado. Perfeito! É realmente isso, o dado sozinho não tem significado. Portanto, po-demos dizer que dado é uma parte da informação, ou seja, a forma mais bruta, sem qualquer semântica. Entretanto, temos a necessidade de entender o conceito de informação, com isso, faremos mais uma atividade: com seu bloco de notas aberto, após a palavra "Jorge", digite "aluno". Neste instante, temos um significado: sabemos que Jorge é um aluno. Portanto,

podemos afirmar que informações são significados que buscamos e isso acontece pelo processamento de dados (Figura 1), em que o conjunto deles gera a informação.

Figura nº 1 – Processamento dos dados



Fonte: Autor

Outro conceito importante é o conhecimento. O conhecimento é obtido com a extração, organização e cruzamento de informações, ou seja, o conjunto de informações organizadas gera conhecimentos.

Um dos maiores ativos das empresas é o que chamamos de ativo intangível. Podemos colocar nesse grupo o conhecimento na fabricação de seus produtos e execução de serviços; e, em uma visão analítica, o conhecimento em todos os processos que levam a empresa a alcançar seus objetivos. Traçando uma linha cronológica decrescente (Figura 2), temos que o conhecimento vem de informações, ao passo que informações são extraídas do processamento de dados.

Figura nº 2 – Linha do tempo, conhecimento, informação e dado.



Fonte: Autor

Agora podemos entender por que desde os primeiros computadores, uma das grandes preocupações é quanto ao armazenamento e manipulação de dados. Com o passar dos anos, o volume de dados foi aumentando e surgindo modelos para organizar e manipular esses dados. A seguir, veremos uma breve descrição sobre os modelos de dados trabalhados no decorrer dos anos.

MODELOS DE DADOS

Modelo hierárquico

O modelo hierárquico é um tipo de gerenciador de banco de dados que conecta os registros em forma de árvore e cada tipo de registro tem apenas uma raiz.

Modelo em rede

O modelo em rede é uma extensão do modelo hierárquico. A representação dos dados é feita por meio de uma coleção de registros. Os relacionamentos são feitos por links.

Modelo relacional

O modelo relacional é o modelo mais aceito pelas empresas nos dias atuais. Sua característica é a organização dos dados em tabelas. As tabelas se relacionam entre si por meio de chaves primárias e estrangeiras.

Modelo orientado a objetos

Neste modelo os dados são armazenados na forma de objetos, por meio de estruturas chamadas classes. Os campos são instâncias dessas classes.

SGBD

Sistema Gerenciador de Banco de Dados ou SGBD é um conjunto de ferramentas baseados em uma linguagem, usadas para gerir a criação da estrutura, exclusão e alteração de bases de dados, tabelas e manipulação de dados.

BIG DATA

A cada dia é gerada uma grande quantidade e variedade de dados. Podemos citar como algumas das empresas que convivem diariamente com esse dilúvio de informações as companhias aéreas, operadoras de telefonia, redes sociais, busca on-line e redes varejistas. No entanto, não basta ter as informações armazenadas, é necessário gerar conhecimento para nortear processos e

tomadas de decisões, ou seja, saber fazer o melhor uso dos dados. Com isso, entramos no conceito de Big Data.

De acordo com Alecrim (2013), inicialmente podemos definir o conceito de Big Data como sendo conjuntos de dados extremamente amplos e que, por este motivo, necessitam de ferramentas especialmente preparadas para lidar com grandes volumes, de modo que toda e qualquer informação nestes meios possa ser encontrada, analisada e aproveitada em tempo hábil. Simplificando, a análise de grandes quantidades de dados para a geração de resultados importantes que, em volumes menores, dificilmente seriam alcançados.

Diariamente são feitas milhões de transações bancárias; milhões de e-mails são enviados ao redor do mundo; redes sociais com um volume cada vez maior de publicações. Portanto, podemos assumir que Big Data é um problema de um imenso volume e variedade de dados. Os bancos de dados relacionais, por características são normalizados, suas consultas possuem muitos joins pelo grande número de tabelas relacionadas, influenciando diretamente na performance da aplicação. Não estou apresentando o fim dos bancos de dados relacionais, apenas estou relacionando ao problema Big Data. Muitas empresas usam e continuarão usando suas bases relacionais. Buscamos, aqui, uma alternativa melhor para o trabalho com grandes volumes de dados. Os bancos de dados tradicionais, principalmente os relacionais, não possuem soluções adequadas a este problema. Ainda, segundo Alecrim (2013), isso acontece porque bancos de dados relacionais normalmente se baseiam em quatro propriedades que tornam a sua adoção segura e eficiente, razão pela qual soluções do tipo são tão populares: atomicidade, consistência, isolamento e durabilidade. Esta combinação é conhecida como ACID (do inglês, Atomicity, Consistency, Isolation e Durability). Neste ponto entra em cena o conceito de NoSQL.

NoSQL

NoSQL (não somente SQL) não faz referência a movimentos contra SQL, refere-se à solução de banco de dados que possibilita o armazenamento de várias formas não se limitando ao modelo relacional. Embora não seja um conceito novo, surgiu em 1998, ainda não se tornou bem conhecido dos profissionais de TI.

De acordo com Oliveira (2013):

> "As análises em grandes massas de dados podem trazer respostas que antes não seriam possíveis de ser obtidas, por isso é grande o potencial de mercado do Big Data, bem como o desejo das empresas de adotá-lo.
>
> Para tratar uma imensa quantidade de dados e aproveitá-los da melhor maneira possível, estão sendo criadas tecnologias que sustentam o Big Data como o NoSQL para infraestrutura de banco de dados, Stream Computing como novo paradigma e Hadoop e MapReduce voltado para análise de dados.
>
> O NoSQL (*Not only Structured Query Language*) é um termo genérico para uma classe definida de bancos de dados não-relacionais, que tem uma propriedade chamada BASE (*Basically Available, Soft state, Eventual consistency*), que distribui os dados em diferentes repositórios tornando-os sempre disponíveis, não se preocupa com a consistência de uma transação, delegando essa função para a aplicação, porém sempre garante a consistência dos dados em algum momento futuro à transação."

NoSQL é uma forma de organizar os dados diferente do modelo relacional. Até pouco tempo atrás só existiam bancos de dados relacionais; se você tivesse qualquer problema teria que buscar uma solução relacional.

Existe uma nova onda de aplicações de bancos de dados, a qual você não olha mais para o passado, você olha para o presente, ou seja, os dados estão chegando, estão sendo armazenados, processados, extraídos os conhecimentos para que possamos recomendar ao cliente.

Veja, na Tabela 1, o comparativo entre bancos de dados SQL e NoSQL.

Tabela nº 1 – Comparativo entre bancos de dados SQL e NoSql.

	Banco de dados SQL	Bancos de dados NoSQL
Modelo de dados	O modelo relacional normaliza dados em estruturas tabulares conhecidas como tabelas, que consistem em linhas e colunas. Um schema define estritamente as tabelas, colunas, índices, relações entre tabelas e outros elementos do banco de dados.	Bancos de dados não relacionais (NoSQL) normalmente não aplicam um schema. Geralmente, uma chave de partição é usada para recuperar valores, conjuntos de colunas ou documentos semiestruturados JSON, XML ou outros que contenham atributos de itens relacionados.

	Banco de dados SQL	Bancos de dados NoSQL
Propriedades ACID	Sistemas de gerenciamento de bancos de dados relacionais (RDBMS) tradicionais são compatíveis com um conjunto de propriedades definido pela sigla ACID: Atomicidade, Constância, Isolamento e Durabilidade. Atomicidade significa "tudo ou nada", ou seja, uma transação é concluída integralmente ou não. Constância significa que quando uma transação é realizada, os dados devem estar em conformidade com o schema do banco de dados. Isolamento exige que as transações simultâneas sejam executadas separadas uma da outra. Durabilidade é a capacidade de se recuperar de uma falha do sistema ou falta de energia inesperada para o último estado conhecido.	Bancos de dados NoSQL normalmente trocam algumas propriedades ACID de sistemas de gerenciamento de bancos de dados relacionais (RDBMS) por um modelo de dados mais flexível que escala horizontalmente. Essas características fazem dos bancos de dados NoSQL uma excelente opção em situações em que os RDBMS deparam com desafios de arquitetura e precisam solucionar uma combinação de gargalos de desempenho, escalabilidade, complexidade operacional e custos crescentes de administração e suporte.
Desempenho	O desempenho normalmente depende do subsistema do disco. A otimização de consultas, índices e estrutura de tabela é necessária para alcançar máximo desempenho.	Desempenho geralmente é uma função do tamanho do cluster do hardware subjacente, da latência de rede e da aplicação que faz a chamada.
Escala	Mais fácil de aumentar a escala "verticalmente" com hardware mais rápido. Outros investimentos são necessários para tabelas relacionais para abranger um sistema distribuído.	Projetado para aumentar a escala "horizontalmente" usando clusters distribuídos de hardware de baixo custo para aumentar a transferência sem aumentar a latência.
APIs	As solicitações para armazenar e recuperar dados são comunicadas usando consultas compatíveis com structured query language (SQL). Essas consultas são analisadas e executadas por sistemas de gerenciamento de bancos de dados relacionais (RDBMS).	APIs baseadas em objetos permitem que desenvolvedores de aplicações armazenem e restaurem facilmente estruturas de dados na memória. As chaves de partição permitem que os aplicativos procurem pares de chave-valor, conjuntos de colunas ou documentos semiestruturados contendo objetos e atributos de aplicativos serializados.
Ferramentas	Os bancos de dados SQL normalmente oferecem um rico conjunto de ferramentas para simplificar o desenvolvimento de aplicações orientadas ao banco de dados.	Oferecem ferramentas para gerenciar clusters e escalabilidade. As aplicações são a interface principal com os dados subjacentes.

Fonte: https://aws.amazon.com/pt/nosql/

Principais características dos bancos de dados NoSQL

Os bancos de dados NoSQL possuem características importantes que os diferenciam dos bancos de dados relacionais. De acordo com Cavalcante (2012), as características dos bancos de dados relacionais são:

- Escalabilidade Horizontal: na medida em que o volume de dados cresce, aumenta-se a necessidade de escalabilidade e melhoria do desempenho. Dentre todas as possibilidades para esta solução, a escalabilidade horizontal se torna a mais viável, porém requer diversas threads ou que processos de um tarefa sejam criadas e distribuídas. Dessa forma, o uso de um banco de dados relacional poderia ser muito complexo. Não queremos dizer que os bancos de dados relacionais não escalam, a verdade é que eles não escalam facilmente. Isto por que no momento em que diversos processos se conectam simultaneamente em um mesmo conjunto de dados há uma geração de uma alta concorrência aumentando assim o tempo de acesso às tabelas. Neste contexto, uma grande vantagem dos bancos NoSQL é justamente a ausência de bloqueios, o que permite a escalabilidade horizontal com uma maior facilidade e eficiência (ele não é afetado pelo aumento da concorrência). Uma alternativa muito utilizada para alcançar a escalabilidade horizontal é o Sharding, que divide os dados em múltiplas tabelas a serem armazenadas ao longo de diversos nós na rede. O que esta técnica faz, na realidade, é romper a cadeia de relacionamentos, que é uma forte característica nos bancos relacionais. É possível realizar o Sharding em banco de dados relacionais de forma manual. Entretanto, esta não é uma tarefa simples e demonstra complexidade de implementação para a equipe que está desenvolvendo.

- Ausência de esquema (Schema-free) ou esquema flexível: Outra característica notável em bancos de dados NoSQL é a ausência parcial ou total de esquema que define a estrutura de dados. É justamente essa ausência de esquema que facilita uma alta escalabilidade e alta disponibilidade, mas em contrapartida não há a garantia de integridade dos dados, fato este que não ocorre no Modelo Relacional.

- Suporte nativo a replicação: Esta é outra forma de prover a escalabilidade, pois, no momento em que permitimos a replicação de forma nativa o tempo gasto para recuperar informações é reduzido.

- API simples para acessar o banco de dados: Em banco de dados NoSQL, o foco não está no armazenamento dos dados e sim como recuperar estes dados de forma eficiente. Pensando nisso, é fundamental APIs

desenvolvidas para facilitar o acesso às devidas informações para que se possa usar o banco de dados de forma rápida e eficiente.

- Consistência eventual: Outra característica particular de bancos NoSQL é que nem sempre a consistência dos dados é mantida. Esta característica tem embasamento no teorema CAP (Consistency, Availability e Partition tolerance) que afirma que em um dado momento só é possível garantir duas destas três propriedades, que seriam consistência, disponibilidade e tolerância à partição. No mundo real, normalmente estas duas últimas são privilegiadas. Como consequência disto, as propriedades do ACID não são respeitadas simultaneamente, ao contrário disto, temos outro conjunto de projetos denominado BASE (Basicamente disponível, estado leve e consistente em momento indeterminado). Ou seja, é necessário haver um planejamento para que o sistema possa tolerar inconsistências temporárias com o objetivo de priorizar a disponibilidade.

Agora que falamos brevemente sobre as principais características nos bancos de dados NoSQL, é importante ressaltar algumas técnicas utilizadas para a implementação de suas funcionalidades. Entre elas estão:

- Map/reduce: permite a manipulação de enormes volumes de dados ao longo de nós em uma rede. Funciona da seguinte forma: na fase map, os problemas são particionados em pequenos problemas que são distribuídos em outros nós na rede. Quando chegam à fase reduce, esses pequenos problemas são resolvidos em cada nó filho e o resultado é pas-sado para o pai, que sendo ele consequentemente filho, repassaria para o seu, até chegar à raiz do problema.

- Consistent hashing: suporta mecanismos de armazenamento e recuperação, onde a quantidade de sites está em constante mudança. É interessante usar essa técnica, pois ela evita que haja uma grande migração de dados entre estes sites, que podem ser alocados ou desalocados para a distribuição dos dados.

- MVCC (Multiversion concurrency control): Oferece suporte a transações paralelas em banco de dados. Por não fazer uso de locks para controle de concorrência, faz com que transações de escrita e leitura sejam feitas simultaneamente.

- Vector clocks: Ordenam eventos que ocorreram em um sistema. Como existe a possibilidade de várias operações estarem acontecendo simultaneamente, o uso de um log de operações informando suas datas se faz importante para informar qual versão de um dado é a mais atual.

Tipos de bancos de dados NoSQL

- Sistemas baseados em armazenamento chave valor
- Sistemas baseados em grafos
- Sistemas orientados a documentos
- Sistemas orientados a colunas

A Tabela 2 apresenta uma descrição dos tipos de bancos de dados NoSQL.

Tabela nº 2 – Comparativo entre bancos de dados SQL e NoSql.

Tipos de bancos de dados NoSQL	
Bancos de dados colunares	Os bancos de dados colunares são otimizados para colunas de leitura e gravação, ao contrário das linhas de dados. O armazenamento orientado a colunas para tabelas do banco de dados é um fator importante no desempenho de consulta analítica, pois ele reduz drasticamente os requisitos gerais de E/S e diminui a quantidade de dados que você precisa carregar do disco.
Bancos de dados de documentos	Os bancos de dados de documentos são projetados para armazenar dados como documentos, geralmente em formato JSON ou XML. Diferentemente dos bancos de dados relacionais tradicionais, o esquema de cada documento não relacional (NoSQL) pode variar, dando a você mais flexibilidade ao organizar e armazenar dados do aplicativo e ao reduzir o armazenamento exigido para valores opcionais.
Bancos de dados de gráficos	Os bancos de dados de gráficos armazenam vértices e links direcionados chamados de bordas. Gráficos podem ser construídos em bancos de dados relacionais (SQL) e não relacionais (NoSQL). Vértices e bordas podem ter propriedades associadas a eles.
Armazéns em memória de chave-valor	As solicitações para armazenar e recuperar dados são comunicadas usando Os armazéns em memória de chave-valor são bancos de dados NoSQL otimizados para cargas de trabalho de aplicativos de leitura pesada (como redes sociais, jogos, compartilhamento de mídia, e portais de P e R) ou cargas de trabalho com uso intenso da computação (como um mecanismo de recomendação). O armazenamento em cache na memória melhora o desempenho do aplicativo ao armazenar pedaços críticos de dados na memória para acesso de baixa latência.

Fonte: https://aws.amazon.com/pt/nosql/

NoSQL database exemplos:

- Google bigtable
- Amazon Dynamo
- Facebook Cassandra
- Apache HBASE
- Linkedin Valdemort

REFERÊNCIAS

ALECRIM, Emerson. **O que é Big Data?** Infowester, 2013. Disponível em: <https://www.infowester.com/big-data.php>. Acesso em: 20 mar. 2017.

OLIVEIRA, Claudio Alves de. **Bancos de Dados NoSQL**. IBM, 2013. Disponível em: <https://www.ibm.com/developerworks/community/blogs/tlcbr/entry/bancos_de_dados_nosql?lang=en>. Acesso em: 20 mar. 2017.

AMAZON WEBSERVICE. **O que é NoSQL?** [S.l]. Disponível em: <https://aws.amazon.com/pt/nosql/>. Acesso em: 20 mar. 2017.

NoSQL. [S.l]. Disponível em: <http://nosql-database.org/>. Acesso em: 20 mar. 2017.

CAVALCANTE, Marília. **Banco de dados NoSQL**: Um novo paradigma - Revista SQL Magazine 102.Devmedia, a.9,n. 07, ago. 2012. Disponível em: <http://www.devmedia.com.br/banco-de-dados-nosql-um-novo-paradigma-revista-sql-magazine-102/25918>. Acesso em: 20 mar. 2017.

Conteúdo:



Dica do Professor: Analisar e compreender a utilização do Banco de Dados NoSQL

O banco de dados NoSQL é uma nova alternativa para solução de problemas relacionados a grandes volumes de dados, quando há a exigência de performance. Saiba mais no vídeo da Dica do Professor.

As imagens do vídeo a seguir possuem audiodescrição. Para acessar o recurso, clique aqui

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Atividade 1 (A1)

1) **Com relação a bancos de dados NoSQL, marque a afirmativa correta.**

A) Veio para substituir os bancos de dados relacionais.

B) Os banco de dados NoSQL têm como característica a organização dos dados em tabelas.

C) O Modelo NoSQL é um tipo de gerenciador de banco de dados que conecta os registros em forma de árvore e cada tipo de registro tem apenas uma raiz.

D) NoSQL (não somente SQL), refere-se à solução de banco de dados que possibilita o armazenamento de uma grande variedade de dados existentes, não se limitando apenas a modelos relacionais.

E) As empresas que optam pelo uso de NoSQL devem eliminar o trabalho com outros modelos de bancos de dados.

2) **Marque a alternativa correta referente a bancos de dados NoSQL.**

A) O modelo NoSQL normaliza dados em estruturas tabulares.

B) Sistemas de gerenciamento de bancos de dados NoSQL são compatíveis com um conjunto de propriedades definido pela sigla ACID.

C) Bancos de dados não relacionais (NoSQL) normalmente não aplicam um schema.

D) Bancos de dados NoSQL são mais fáceis de aumentar a escala "verticalmente" com hardware mais rápido.

E) Em bancos de dados NoSQL as solicitações para armazenar e recuperar dados são comunicadas usando consultas compatíveis com structured query language (SQL).

3) **De acordo com as características dos bancos de dados NoSQL, pode-se afirmar que:**

A) A característica ausência de esquema (Schema-free) ou esquema flexível é outra forma de prover a escalabilidade em bancos de dados NoSQL.

B) Ausência de esquema garante a integridade dos dados.

C) Na consistência eventual, o foco não está no armazenamento dos dados e sim como recuperar estes dados de forma eficiente.

D) Map/reduce suporta mecanismos de armazenamento e recuperação, onde a quantidade de sites está em constante mudança.

E) Vector clocks cria um vetor para o armazenamento de dados.

4) Referente aos tipos de bancos de dados NoSQL. Marque a afirmativa correta.

A) Bancos de dados colunares são projetados para armazenar dados como documentos.

B) Chave/Valor são projetados para armazenar dados como documentos.

C) Banco de dados orientado a documentos armazenam vértices e links direcionados chamados de bordas.

D) O banco de dados orientado a documentos armazena pedaços críticos de dados na memória para acesso de baixa latência.

E) Bancos de dados orientado a grafos podem ser construídos em ambos os modelos de bancos de dados, relacionais e NoSQL.

5) Existem muitos bancos de dados para se trabalhar com NoSQL, cada banco está associado a um determinado tipo do NoSQL. Marque a alternativa em que todos os bancos de dados estão associados ao seu respectivo tipo.

A) Baseado em documentos: MongoDB, CouchDB, BigCouch e RavenDB.

B) Baseado em grafos: Cassandra e Hypertable.

C) Baseado em chave-valor: Neo4J e Infinite Graph.

D) Baseado em documentos: Dynamo, Azure Table Storagee e Couchbase Server.

E) Baseado em coluna: Amazon SimpleDB, Cloudata, Cloudera, SciDB, HPCC e Stratosphere.

Saiba mais: Analisar e compreender a utilização do Banco de Dados NoSQL

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

Acompanhe, a seguir, uma entrevista com Eduardo Morelli, coordenador do curso de Pós-graduação em Big Data do Instituto Infnet, do Rio de Janeiro, falando sobre NoSQL e Bigdata.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Você sabe o que é MongoDB e para o que serve? Veja mais clicando no próximo link.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

Neste vídeo você poderá saber mais sobre aplicações com um banco de dados NoSQL.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.